Guía completa de Machine Learning: regresión lineal, polinómica y regularización

Desde cero, con rigor,
para entender, defender y aplicar

Bootcamp The Bridge · Bloque de Machine Learning

Material de estudio — todos los conceptos explicados paso a paso

Índice

Esta guía cubre todo el contenido de los cuatro bloques de Machine Learning que vimos (y que te perdiste) en el bootcamp. Cada concepto se explica primero con palabras corrientes, después con la fórmula matemática, y por último con un ejemplo de código Python para que veas cómo se usa en sklearn. Lee las secciones en orden: cada una se apoya en lo anterior.

Parte I — Fundamentos de Machine Learning

1. Qué es el Machine Learning

2. Regresión lineal: la idea base

2.1 Regresión lineal simple

2.2 Regresión lineal múltiple

2.3 Interpretación de los coeficientes β

3. Cómo encuentra el modelo la "mejor recta": función de costes y Gradient Descent

3.1 La función de costes (MSE)

3.2 Gradient Descent (descenso del gradiente)

3.3 Learning rate

Parte II — Medir el error

4. Residuos: la base de todas las métricas

5. MAE — Error Absoluto Medio

6. MSE — Error Cuadrático Medio

7. RMSE — Raíz del Error Cuadrático Medio

8. MAPE — Error Porcentual Absoluto Medio

9. R^2 — Coeficiente de determinación

10. Correlación de Pearson y p-valor

Parte III — Metodología correcta

11. Train / Test split

12. Overfitting, Underfitting y el dilema sesgo-varianza

13. StandardScaler y MinMaxScaler

14. Validación cruzada (cross_val_score)

15. Condiciones para aplicar regresión lineal

15.1 Linealidad

15.2 Multicolinealidad

15.3 Homocedasticidad

15.4 Independencia y normalidad de los residuos

Parte IV — Regresión polinómica

16. Cuando la relación no es lineal

17. PolynomialFeatures de sklearn

18. Elección del grado óptimo

Parte V — Regularización

19. Por qué regularizar

20. Ridge (L2)

21. Lasso (L1)

22. Elastic Net

23. Cómo elegir entre Ridge, Lasso y Elastic Net

Parte VI — Cheat sheet y glosario

24. Pipeline típico de un problema de regresión

25. Cheat sheet de métricas y modelos

26. Glosario en orden alfabético

Parte I
Fundamentos de Machine Learning

1. Qué es el Machine Learning

Antes de meternos con regresiones, conviene saber dónde estamos. El **Machine Learning** (aprendizaje automático) es una rama de la inteligencia artificial en la que un ordenador aprende patrones a partir de ejemplos, sin que nadie le programe reglas explícitas. En lugar de decirle «si el precio sube, haz esto», le enseñas miles de casos pasados y deja que él solo descubra la relación.

Dentro del Machine Learning hay tres grandes familias:

- **Aprendizaje supervisado**. Tienes datos *etiquetados*: para cada ejemplo conoces la respuesta correcta. El modelo aprende a predecirla. Este es nuestro terreno.
- **Aprendizaje no supervisado**. Tienes datos sin etiquetar y el modelo busca agrupaciones o estructuras (clustering, reducción de dimensionalidad, etc.).
- **Aprendizaje por refuerzo**. Un agente aprende por ensayo y error recibiendo recompensas (videojuegos, robótica).

Dentro del supervisado hay dos tipos de problema según lo que quieras predecir:

- **Regresión**. La respuesta es un número continuo: el precio de una casa, la temperatura mañana, las ventas del próximo mes. Todo lo que vas a ver en esta guía.
- **Clasificación**. La respuesta es una categoría: spam o no spam, perro o gato, diabético o no. Se verá en otro bloque del bootcamp.

> **QUEDÉMONOS CON ESTO**
>
> El Machine Learning supervisado de regresión consiste en: *tengo ejemplos pasados con la respuesta correcta (entrenamiento); el modelo aprende la relación entre las variables; después uso esa relación para predecir respuestas sobre datos nuevos (test).*

2. Regresión lineal: la idea base

2.1 Regresión lineal simple

Imagínate una nube de puntos en un gráfico. En el eje X pones una variable (por ejemplo, las horas que ha estudiado cada alumno) y en el eje Y otra (la nota que ha sacado). Cada alumno es un punto.

La **regresión lineal simple** consiste en dibujar la mejor línea recta posible por esa nube. ¿Para qué? Para que, cuando aparezca un alumno nuevo y te diga cuántas horas ha estudiado, puedas usar la recta para predecir su nota.

La ecuación de una recta es:

$$Y = \beta_0 + \beta_1 \cdot X$$

- **Y** es lo que queremos predecir (la nota). Se llama *variable dependiente*, *target*, *respuesta* o *regresando*.
- **X** es lo que conocemos (las horas de estudio). Se llama *variable independiente*, *feature*, *predictor* o *regresor*.
- $\boldsymbol{\beta_0}$ (se lee «beta sub cero») es el **intercepto** u **ordenada en el origen**. Es el valor de Y cuando X vale 0: el punto por donde la recta cruza el eje vertical.
- $\boldsymbol{\beta_1}$ es la **pendiente** o **coeficiente**: cuánto sube o baja Y por cada unidad que aumenta X.

EJEMPLO NUMÉRICO

Supón que la recta aprendida es *nota = 2 + 0.8 · horas*. Eso significa:

- Un alumno que estudió 0 horas: la nota predicha sería 2 (el intercepto).
- Por cada hora extra, la nota sube 0.8 puntos.
- Un alumno que estudió 5 horas: *2 + 0.8 × 5 = 6*.

2.2 Regresión lineal múltiple

En la vida real rara vez predices algo a partir de una sola variable. El precio de una casa no depende solo de los metros cuadrados, también del barrio, el número de habitaciones, la antigüedad, etc. Cuando hay varias features, la ecuación se extiende:

$$Y = \beta_0 + \beta_1 \cdot X_1 + \beta_2 \cdot X_2 + \beta_3 \cdot X_3 + \ldots + \beta_j \cdot X_j$$

Cada feature tiene su propio coeficiente β. En la diapositiva que vio tu clase, el ejemplo era:

$$\text{Sales} = 500 + 50 \cdot \text{TV} + 20 \cdot \text{radio} + 15 \cdot \text{newspaper}$$

Geométricamente, en vez de una recta tenemos un **hiperplano** en un espacio de tantas dimensiones como features. No lo podemos dibujar si hay más de 2 features, pero la matemática es la misma.

2.3 Interpretación de los coeficientes β

Ésta es una parte que te van a preguntar seguro en cualquier entrevista o examen. Los coeficientes β no son cualquier cosa: **tienen significado real**.

> **LA REGLA DE ORO PARA INTERPRETAR B_J**
>
> β_j es *cuánto varía Y cuando la feature X_j aumenta en 1 unidad, manteniendo el resto de features constantes.*
>
> Si β_j es positivo, cuando X_j sube, Y sube. Si es negativo, cuando X_j sube, Y baja.

Aplicado al ejemplo de ventas:

- β_{TV} = 50: por cada euro extra invertido en TV (manteniendo radio y prensa igual), las ventas suben 50.
- β_{radio} = 20: por cada euro extra en radio, las ventas suben 20.
- $\beta_{newspaper}$ = 15: por cada euro extra en prensa, las ventas suben 15.

β_0 = 500 sería lo que se vendería si no invirtiéramos nada en publicidad. A menudo el intercepto no tiene una interpretación útil por sí solo (a veces esas condiciones son absurdas, como «una casa de 0 m²»), pero matemáticamente existe.

> **OJO: LOS B SIN ESCALAR NO REFLEJAN IMPORTANCIA**
>
> Podrías pensar: «β_{TV} = 50 es mayor que β_{radio} = 20, luego TV es más importante». Eso es un error típico. Los coeficientes están en las unidades de cada variable, y comparar coeficientes de variables en escalas distintas no tiene sentido (por ejemplo, comparar euros con número de habitaciones). Para comparar la importancia hay que estandarizar los datos primero — lo vemos en la sección del *StandardScaler*.

3. Cómo encuentra el modelo la "mejor recta": función de costes y Gradient Descent

Hasta ahora hemos hablado del modelo como si la recta apareciese por arte de magia. Pero el ordenador tiene que calcular los β óptimos de algún modo. Este apartado es el que tu profesor dio en clase y que te perdiste — es la mecánica interna del aprendizaje.

3.1 La función de costes (MSE)

Por una nube de puntos pueden pasar infinitas rectas. ¿Cuál es la «mejor»? Necesitamos un criterio numérico que diga qué recta se ajusta mejor a los datos. Ese criterio se llama **función de costes** o *loss function*.

Para la regresión lineal, la función de costes estándar es el **Error Cuadrático Medio** (MSE, *Mean Squared Error*). Se define como la media de los errores al cuadrado:

$$\mathrm{MSE} = (1/\mathrm{N}) \cdot \Sigma\ (\mathrm{Y_i} - \hat{\mathrm{Y}}_\mathrm{i})^2$$

Donde:

- **N** es el número de observaciones.
- $\mathbf{Y_i}$ es el valor real del i-ésimo ejemplo.
- $\hat{\mathbf{Y}}_\mathbf{i}$ (léase «Y sombrero sub i») es el valor predicho por el modelo para ese ejemplo.
- $\mathbf{(Y_i - \hat{Y}_i)}$ es el error o residuo de ese ejemplo (en el capítulo 4 profundizamos).

> **POR QUÉ AL CUADRADO**
>
> Si sumásemos los errores tal cual, los positivos y los negativos se cancelarían entre sí y el total daría casi cero aunque la recta fuera horrible. Al elevar al cuadrado conseguimos dos cosas: todos los errores se vuelven positivos, y los errores grandes pesan mucho más que los pequeños (un error de 4 aporta 16, uno de 2 aporta solo 4). Esto es deseable porque normalmente queremos evitar equivocaciones grandes a toda costa.

Así que nuestro objetivo matemático es claro: **encontrar los valores de β_0, β_1, ..., β_j que minimizan la función de costes MSE**. Es un problema de optimización.

3.2 Gradient Descent (descenso del gradiente)

Para problemas simples podríamos calcular los β óptimos directamente con una fórmula cerrada (*Ordinary Least Squares*, OLS). Pero cuando hay muchas features o modelos más complejos, no es práctico. Usamos un método iterativo llamado **Gradient Descent**.

> **LA METÁFORA DE LA MONTAÑA CON NIEBLA**
>
> Imagínate que estás en una montaña rodeado de niebla espesa y quieres llegar al valle. No ves a lo lejos. ¿Qué haces? Miras la pendiente del suelo justo donde pisas, y das un paso en la dirección más descendente. Repites. Paso a paso, sin ver lejos, vas bajando hasta que el suelo deja de descender: has llegado al valle. El Gradient Descent hace exactamente eso con la función de costes.

Matemáticamente, la «pendiente del suelo» se calcula con el **gradiente**, que es la derivada parcial de la función de costes respecto a cada parámetro β. El gradiente te dice hacia dónde crece la función; nosotros queremos lo contrario, así que caminamos en dirección opuesta (de ahí «descenso»).

El algoritmo es así:

1. Empezamos con valores aleatorios de β.
2. Calculamos la función de costes (MSE) con esos β.
3. Calculamos el gradiente (las derivadas parciales de la función de costes respecto a cada β).
4. Actualizamos cada β restando una pequeña fracción del gradiente: $\beta_{nuevo} = \beta_{viejo} - \alpha \cdot \text{gradiente}$.
5. Volvemos al paso 2. Iteramos hasta que el MSE deja de bajar (convergencia).

La intuición visual con una curva $f(x) = x^2$ es muy útil:

- Si estoy a la derecha del mínimo, la pendiente es positiva → tengo que ir hacia la izquierda (restar del X actual).
- Si estoy a la izquierda del mínimo, la pendiente es negativa → tengo que ir hacia la derecha (sumar al X actual).
- Si la pendiente es cero, estoy en el mínimo → he convergido.

El truco es que al restar α · gradiente, el signo ya se encarga automáticamente: si el gradiente es positivo, resto; si es negativo, al restar un número negativo sumo.

3.3 Learning rate (α)

El parámetro α (alfa) que aparece en la fórmula $\beta_{nuevo} = \beta_{viejo} - \alpha \cdot gradiente$ se llama **learning rate** o tasa de aprendizaje. Controla el tamaño de cada paso.

- Si α es **demasiado pequeño**: avanzas a pasitos minúsculos. Tardas muchísimo en llegar al mínimo (miles de iteraciones).
- Si α es **demasiado grande**: das zancadas gigantes. Puedes pasarte de largo del mínimo y, en el peor de los casos, oscilar sin converger nunca, o incluso divergir y mandar el modelo al infinito.
- Si α es **apropiado**: avanzas decididamente al principio y vas afinando cerca del mínimo.

El learning rate es un **hiperparámetro**: lo decides tú, no lo aprende el modelo. Valores típicos van de 0.001 a 0.1. En la regresión lineal de sklearn esto te da igual porque sklearn resuelve directamente con la fórmula cerrada, pero en otros modelos (redes neuronales) el learning rate es crítico.

VOCABULARIO A RETENER

Función de costes (loss function)

Métrica que cuantifica el error del modelo. Para regresión lineal, el MSE.

Gradient Descent

Algoritmo iterativo que busca los parámetros que minimizan la función de costes.

Gradiente

Vector de derivadas parciales. Indica la dirección en la que crece más rápido la función.

Learning rate (α)

Tamaño del paso en cada iteración del Gradient Descent.

Convergencia

Cuando el algoritmo deja de mejorar: el modelo ha encontrado el mínimo (o se ha estancado).

Hiperparámetro

Configuración del algoritmo que decides tú antes de entrenar (learning rate, alpha de Ridge, grado del polinomio…). No lo aprende el modelo de los datos.

Parámetro

Valores que el modelo aprende durante el entrenamiento (los β en regresión lineal).

Parte II
Medir el error

Una vez entrenado el modelo, hay que saber si es bueno o malo. Para eso existen varias métricas. Cada una mide algo ligeramente distinto y tiene sus pros y sus contras. Esta parte es imprescindible: todo el vocabulario MAE, MSE, RMSE, MAPE, R^2 que aparece mil veces en los notebooks vive aquí.

4. Residuos: la base de todas las métricas

Cuando el modelo hace una predicción, casi nunca acierta exactamente el valor real. Hay una diferencia entre lo que predijo y lo que realmente pasó. Esa diferencia se llama **residuo** o **error**:

$$\text{error}_i = Y_i - \hat{Y}_i$$

Donde Y_i es el valor real y $\hat{Y}_i$ es la predicción del modelo para el ejemplo i.

- Si el residuo es **positivo**, el modelo ha *subestimado* (el real es mayor que lo predicho).
- Si es **negativo**, el modelo ha *sobreestimado* (ha predicho más de lo que era).
- Si es **cero**, ha acertado ese punto exactamente.

Cada ejemplo del conjunto de test tiene su propio residuo. El problema es resumir todos esos residuos en un único número que nos diga «cómo de bueno es mi modelo». De ahí nacen las métricas.

5. MAE — Error Absoluto Medio

> **QUÉ ES EN UNA FRASE**
>
> En promedio, ¿cuánto se equivoca mi modelo en cada predicción, sin importar si se pasa o se queda corto?

Fórmula:

$$\text{MAE} = (1/N) \cdot \Sigma\, |Y_i - \hat{Y}_i|$$

Tomamos el valor absoluto de cada error (para que todos sean positivos y no se cancelen entre sí), los sumamos y los dividimos por el número de ejemplos. El resultado está en las mismas unidades que el target.

EJEMPLO NUMÉRICO

Si mis errores son +3, −2, +4, −1:

1. Valores absolutos: 3, 2, 4, 1
2. Suma: 3 + 2 + 4 + 1 = 10
3. Media: 10 / 4 = **2.5**

MAE = 2.5. Interpretación: «en promedio, mi modelo se equivoca en 2.5 unidades en cada predicción».

Ventaja: es el más fácil de interpretar. Si predices precios en euros, el MAE te dice «me equivoco de media en X euros».

Inconveniente: trata a todos los errores por igual. Un error grande y uno pequeño pesan proporcionalmente. Si en tu aplicación los errores grandes son especialmente graves (por ejemplo, predicciones médicas), puede ser insuficiente.

Cómo se calcula en sklearn:

```
from sklearn.metrics import mean_absolute_error
mae = mean_absolute_error(y_test, y_pred)
```

6. MSE — Error Cuadrático Medio

QUÉ ES EN UNA FRASE

Como el MAE, pero elevamos los errores al cuadrado antes de promediar. Esto castiga mucho más los errores grandes.

Fórmula:

$$\mathrm{MSE} = (1/\mathrm{N}) \cdot \Sigma\ (\mathrm{Y}_{\mathrm{i}} - \hat{\mathrm{Y}}_{\mathrm{i}})^2$$

EJEMPLO NUMÉRICO

Con los mismos errores +3, −2, +4, −1:

1. Al cuadrado: 9, 4, 16, 1
2. Suma: 30
3. Media: 30 / 4 = **7.5**

MSE = 7.5. Fíjate en la diferencia: el error de 4 aporta 16 al MSE (frente a 4 al MAE). Eleva mucho el resultado.

Ventaja: castiga errores grandes. Útil cuando en tu problema esos errores son especialmente costosos.

Inconveniente grave: las unidades están al cuadrado. Si predices precios en euros, el MSE está en «euros al cuadrado», que no significa nada intuitivo. Por eso casi nunca se reporta solo: se reporta su raíz (el RMSE).

En sklearn:

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_test, y_pred)
```

7. RMSE — Raíz del Error Cuadrático Medio

QUÉ ES EN UNA FRASE

El MSE al que le aplicamos la raíz cuadrada. Así recuperamos las unidades originales.

Fórmula:

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Es lo mejor de los dos mundos: castiga errores grandes (como el MSE) pero es interpretable en las unidades del target (como el MAE). Por eso es la métrica más usada para reportar.

COMPARAR MAE CON RMSE ES DIAGNÓSTICO

El RMSE siempre es mayor o igual que el MAE. La diferencia entre ambos te chiva cómo están distribuidos los errores:

- Si MAE y RMSE son parecidos: los errores son uniformes, ninguno especialmente grande.
- Si RMSE es mucho mayor que MAE: hay unos pocos errores muy grandes (outliers) que están tirando del RMSE hacia arriba.

sklearn no tiene función `root_mean_squared_error` directa (depende de la versión). Lo típico es:

```
import numpy as np
from sklearn.metrics import mean_squared_error
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

8. MAPE — Error Porcentual Absoluto Medio

QUÉ ES EN UNA FRASE

El error expresado como porcentaje del valor real. Útil cuando el target varía mucho en escala y quieres saber el error relativo.

Fórmula:

$$\mathrm{MAPE} = (1/N) \cdot \Sigma \mid (Y_i - \hat{Y}_i) \;/\; Y_i \mid \times 100$$

EJEMPLO QUE ACLARA POR QUÉ EXISTE EL MAPE

Tenemos dos pacientes con el mismo error absoluto:

Paciente	Real	Predicho	Error abs	Error %
A	200	190	10	5%
B	50	40	10	20%

El error absoluto es el mismo (10), pero fallar 10 sobre un real de 200 no es tan grave como fallar 10 sobre un real de 50. El MAPE lo captura.

Ventaja: es intuitivo (todo el mundo entiende «me equivoco un 7%») y es independiente de la escala, útil para comparar modelos entre problemas distintos.

Inconvenientes importantes:

- Si algún Y real vale 0, la división explota (no se puede dividir entre cero).
- Si algún Y real es muy pequeño, el porcentaje de ese caso puede ser enorme y distorsionar la media.
- No es simétrico: penaliza más subestimar que sobreestimar.

En sklearn:

```
from sklearn.metrics import mean_absolute_percentage_error
mape = mean_absolute_percentage_error(y_test, y_pred) * 100  # multiplicamos
por 100 para expresarlo en %
```

Nota: sklearn devuelve una proporción (0.07 significa 7%), por eso multiplicamos por 100.

9. R^2 — Coeficiente de determinación

Esta es la métrica más reveladora. MAE, MSE, RMSE y MAPE te dicen *cuánto* te equivocas en términos absolutos o relativos. Pero el **R^2** te dice algo más profundo: *si tu modelo sirve para algo o no*.

La idea del baseline tonto

Imagina que no tienes ningún modelo. ¿Cuál es la predicción más tonta que podrías hacer? **Responder siempre la media**. Si la nota media de una clase es 4.2, pues dices 4.2 para todos, sin mirar nada. Es el baseline más simple y más pobre posible.

El R^2 compara tu modelo con ese baseline tonto:

Valor de R^2	Significa
$R^2 = 1$	Modelo perfecto. Acierta todo sin error.
$R^2 = 0.8$	El modelo explica el 80% de la variabilidad del target.
$R^2 = 0.5$	El modelo explica la mitad. Decente según contexto.
$R^2 = 0$	Tu modelo predice igual de bien que responder siempre la media. No aporta nada.
$R^2 < 0$ (negativo)	Tu modelo predice peor que responder siempre la media. Has hecho un modelo que perjudica.

Fórmula:

$$R^2 = 1 - (SS_{res} / SS_{tot})$$

Donde:

- **SS_{res}** (Residual Sum of Squares): $\Sigma(Y_i - \hat{Y}_i)^2$. La suma de los errores al cuadrado del modelo.
- **SS_{tot}** (Total Sum of Squares): $\Sigma(Y_i - \bar{y})^2$. La suma de los errores al cuadrado del baseline tonto (predecir siempre la media $\bar{y}$).

POR QUÉ LA FÓRMULA TIENE SENTIDO

SS_{res} / SS_{tot} es la fracción del error que NO has conseguido eliminar respecto al baseline. Si tu modelo elimina el 80% del error, $SS_{res}/SS_{tot} = 0.2$, y $R^2 = 1 - 0.2 = 0.8$. Si tu modelo es igual que el baseline, esa fracción es 1, y $R^2 = 0$. Si tu modelo es peor ($SS_{res} > SS_{tot}$), la fracción supera 1 y R^2 se vuelve negativo.

EL R^2 NEGATIVO ES UNA ALARMA ROJA

Si entrenas un modelo y el R^2 en test sale negativo, el modelo *no sirve para nada* — literalmente, lo habrías hecho mejor ignorando los datos y respondiendo siempre la media. Esto pasa típicamente cuando las features no tienen relación real con el target (véase el ejercicio de Height: edad no predice altura en adultos) o cuando el modelo está muy mal calibrado.

En sklearn se calcula de dos formas equivalentes:

```
# Opción 1: método score del modelo
r2 = modelo.score(X_test, y_test)

# Opción 2: función de métricas
from sklearn.metrics import r2_score
r2 = r2_score(y_test, y_pred)
```

10. Correlación de Pearson y p-valor

Estas dos herramientas no son métricas del modelo propiamente, sino del análisis exploratorio previo. Te ayudan a saber si tiene sentido intentar una regresión lineal o no.

10.1 Correlación de Pearson

La **correlación de Pearson** es un número entre −1 y +1 que te dice si dos variables tienden a moverse juntas de forma lineal.

Valor de r	Interpretación
r = +1	Correlación positiva perfecta: cuando X sube, Y sube en línea recta.
r ≈ +0.7	Correlación positiva fuerte (p. ej. altura y peso de adultos).
r ≈ +0.3	Correlación positiva débil.
r = 0	No hay relación lineal.
r ≈ −0.3	Correlación negativa débil.
r = −1	Correlación negativa perfecta: cuando X sube, Y baja en línea recta.

UN DATO MUY ÚTIL: R AL CUADRADO

Si elevas la correlación al cuadrado (r^2) obtienes el porcentaje de variabilidad de Y que explica X linealmente. Si r = 0.7, entonces $r^2 = 0.49$: la variable X explica el 49% de la variabilidad de Y.

Esta r^2 en un problema con una sola feature coincide con el R^2 del modelo de regresión lineal. ¡No es casualidad!

10.2 Limitaciones de Pearson

- Solo detecta relaciones **lineales**. Si dos variables tienen relación en forma de U o curva, Pearson puede dar casi cero aunque la relación exista (ahí entra la regresión polinómica).
- **Correlación no implica causalidad**. El ejemplo clásico: el número de cigüeñas y el número de nacimientos en un país correlacionan positivamente. Pero no es que las cigüeñas traigan bebés: ambas cosas correlacionan con variables rurales/urbanas. Siempre que veas correlación, pregúntate qué variable oculta puede estar detrás.

10.3 P-valor: ¿es fiable esta correlación?

Cuando tienes pocos datos, cualquier correlación que midas podría deberse al puro azar. Necesitas una herramienta que te diga si lo que ves es señal real o ruido. Esa herramienta es el **p-valor**.

QUÉ MIDE EL P-VALOR

El p-valor es la probabilidad de observar una correlación como la que tienes (o más extrema) *si en la realidad no hubiera ninguna relación entre X e Y*.

La convención es:

- **p-valor < 0.05:** poco probable que sea casualidad → la correlación se considera *estadísticamente significativa*.

- **p-valor ≥ 0.05:** podría ser casualidad perfectamente → *no puedes afirmar* que haya relación.

El umbral de 0.05 es una convención arbitraria, no una ley física. Algunos estudios usan 0.01 o 0.001 para ser más estrictos.

POR QUÉ DEPENDE DEL TAMAÑO DE LA MUESTRA

Con 20 observaciones, una correlación de 0.25 probablemente tenga $p \approx 0.3$ (no significativa). Con las mismas variables y 1000 observaciones, la misma correlación tendrá $p < 0.001$ (muy significativa). Más datos = más capacidad para detectar relaciones reales.

Cálculo con scipy:

Parte III
Metodología correcta

Entrenar un modelo no es solo hacer model.fit(). Si no sigues una metodología correcta, tu modelo puede parecer excelente en tus datos y fracasar estrepitosamente cuando lo pongas a predecir casos nuevos. Esta parte es la que separa a un data analyst profesional del que solo copia notebooks.

11. Train / Test split

11.1 El problema que resuelve

Si entreno mi modelo con todos los datos disponibles y después evalúo el error con esos mismos datos, estoy haciendo trampa. El modelo puede **memorizar** los datos en lugar de aprender patrones generales. En los datos de entrenamiento parecerá que acierta mucho, pero con datos nuevos fallará.

Este fenómeno se llama **overfitting** (sobreajuste) y es enemigo número uno del Machine Learning.

11.2 La solución: dividir los datos

Partimos los datos en dos grupos antes de entrenar nada:

- **Train (entrenamiento):** el modelo los usa para aprender. Suele ser el 70-80% de los datos.
- **Test (prueba):** el modelo no los ve durante el entrenamiento. Después los usamos para evaluar de forma honesta cómo predice en datos que nunca ha visto.

Si el modelo funciona bien en test (no solo en train), sabemos que ha aprendido patrones reales y generaliza.

11.3 En sklearn

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,        # 20% a test, 80% a train
    random_state=42        # semilla aleatoria para que la división sea re-
producible
)
```

Qué hace cada parámetro:

- `test_size=0.2`: proporción de datos que van a test. Valores típicos: 0.2 o 0.3.

- `random_state=42`: fija la semilla del generador aleatorio. Si lo quitas, cada vez que ejecutes saldrá una división distinta; si lo fijas, siempre sale la misma. Esto es útil para reproducir resultados.

> **REGLA INQUEBRANTABLE**
>
> Evalúa **SIEMPRE** en test, nunca en train. Las métricas en train son demasiado optimistas porque el modelo ya vio esos datos. La métrica que realmente importa es la de test.

11.4 Splits alternativos: por posición

A veces el enunciado pide un split distinto. Por ejemplo, «los últimos 20 registros a test». Eso se hace con slicing, no con train_test_split:

```
X_train = X[:-20]   # todo excepto los últimos 20
X_test  = X[-20:]   # los 20 últimos
y_train = y[:-20]
y_test  = y[-20:]
```

Este tipo de split tiene sentido en series temporales (donde no puedes usar el futuro para predecir el pasado), pero en problemas generales suele ser preferible el split aleatorio.

12. Overfitting, Underfitting y el dilema sesgo-varianza

Esta sección es uno de los conceptos más importantes de todo el bloque de ML. Entenderlo bien te va a ahorrar muchísimas horas de frustración.

12.1 Las dos formas de fracasar de un modelo

> **LAS DOS ENFERMEDADES POSIBLES**
>
> **Underfitting (subajuste):** el modelo es demasiado simple para capturar la complejidad de los datos. Falla tanto en train como en test. Como intentar trazar una recta para datos que claramente siguen una curva.
>
> **Overfitting (sobreajuste):** el modelo es demasiado complejo y se ha aprendido los datos de memoria, incluido el ruido aleatorio. Acierta casi todo en train, pero falla en test. Como un alumno que se sabe los exámenes de memoria pero no ha entendido la materia.

12.2 El gráfico clave: error vs complejidad

Hay un gráfico que aparece en toda la bibliografía de ML y que conviene tener mentalmente grabado:

- En el eje X está la **complejidad del modelo** (grado del polinomio, número de features, profundidad del árbol…).

- En el eje Y está el **error**.
- Se dibujan dos curvas: el error en **train** (azul) y el error en **test/validación** (verde).

La forma típica es:

- **Con poca complejidad** (izquierda del gráfico): ambos errores son altos. Estamos en zona de *underfitting*. El modelo es demasiado simple.
- **Con complejidad media**: ambos errores son bajos y parecidos. **Esta es la zona óptima.**
- **Con mucha complejidad** (derecha): el error de train sigue bajando pero el de test empieza a subir. Estamos en zona de *overfitting*.

> **CÓMO DETECTAR OVERFITTING SIN EL GRÁFICO**
>
> Si al comparar métricas obtienes: *R^2 train = 0.95 y R^2 test = 0.60* (gran diferencia), estás en overfitting seguro. El modelo se sabe los datos de train de memoria pero no generaliza.
>
> Si obtienes: *R^2 train = 0.55 y R^2 test = 0.52* (parecidos, ambos mediocres), el problema no es overfitting sino underfitting: el modelo es demasiado simple.

12.3 El dilema sesgo-varianza (bias-variance tradeoff)

Los dos tipos de error se explican con dos conceptos estadísticos:

- **Sesgo (bias):** error por ser demasiado simple. Un modelo con alto sesgo hace suposiciones muy rígidas que no corresponden a la realidad. Underfitting = alto sesgo.
- **Varianza (variance):** error por ser demasiado sensible a los datos concretos de entrenamiento. Un modelo con alta varianza cambia mucho si cambias un poco los datos. Overfitting = alta varianza.

El dilema es que normalmente **reducir uno aumenta el otro**. Un modelo muy simple tiene mucho sesgo y poca varianza; uno muy complejo, poca sesgo pero mucha varianza. El arte es encontrar el punto óptimo.

La regularización (Ridge, Lasso, Elastic Net que veremos en la Parte V) son precisamente técnicas para controlar este equilibrio.

13. StandardScaler y MinMaxScaler

13.1 Por qué escalar

Imagínate que tienes dos features con escalas muy distintas: edad (entre 20 y 50) e ingresos anuales (entre 20.000 y 100.000). Algunos algoritmos se ven afectados por esas escalas. Además, los coeficientes β de la regresión salen en las unidades de cada variable, lo que hace imposible compararlos.

13.2 StandardScaler

Transforma cada feature para que tenga media 0 y desviación típica 1:

$$x_{escalada} = (x - media) \ / \ desviación_típica$$

EJEMPLO

Si la edad media en tu grupo es 32 y la desviación típica 6:

- Alguien de 38 años → (38 − 32)/6 = +1 (una desviación por encima de la media)
- Alguien de 26 años → (26 − 32)/6 = −1
- Alguien de 32 años → 0 (justo en la media)

Ahora la edad está en desviaciones típicas, no en años. Si haces lo mismo con los ingresos, las dos variables están en la misma escala.

13.3 MinMaxScaler

Alternativa al StandardScaler. Transforma cada feature para que sus valores estén entre 0 y 1:

$$x_{escalada} = (x - x_{min}) \ / \ (x_{max} - x_{min})$$

El valor mínimo queda en 0, el máximo en 1, el resto proporcionalmente entre medias.

Cuándo usar qué	Preferible
La distribución es aproximadamente normal	StandardScaler
La distribución no es normal o hay outliers grandes	MinMaxScaler (el StandardScaler se ve más afectado por outliers)
Necesito valores estrictamente en [0,1] (por ejemplo, para redes neuronales)	MinMaxScaler
En duda	StandardScaler por defecto

13.4 El peligro: contaminación train/test

ERROR GRAVÍSIMO Y MUY FRECUENTE

Si aplicas el scaler a *todo el dataset antes de hacer el split*, estás mezclando información de test con train: el modelo «ve» indirectamente estadísticas del test. Tus métricas quedarán infladas y engañosas.

La forma correcta es ajustar el scaler **solo con los datos de train**, y después aplicar la misma transformación a test.

En el notebook de Advertising de tu clase hay un ejemplo muy clarificador:

CASO REAL

Tenemos edades: [20, 35, 40, 25, 18]

Train: [20, 35, 40, 25]. Test: [18].

Incorrecto (scaler sobre todo X):

- Mínimo = 18, máximo = 40
- 18 se escala a 0 (el mínimo)

Correcto (scaler solo sobre train):

- Mínimo train = 20, máximo train = 40
- 18 (en test) se escala a −0.1 — fuera del rango [0,1]

En el mundo real, los datos de producción pueden tener valores fuera del rango de train. Lo correcto es que el scaler se ajuste solo a train y admita que test pueda quedar fuera de [0,1].

13.5 Forma correcta en sklearn

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train)                  # aprende media y desviación SOLO de
train
X_train_scaled = scaler.transform(X_train)
X_test_scaled  = scaler.transform(X_test)   # aplica la misma transformación
a test
```

También existe el atajo `fit_transform` que hace fit y transform en una sola línea:

```
X_train_scaled = scaler.fit_transform(X_train)   # solo con train
X_test_scaled  = scaler.transform(X_test)        # NUNCA fit_transform con
test
```

REGLA MENTAL

fit solo con train. *transform* con train y con test por separado. **Nunca** hagas fit con test.

13.6 ¿Escalar mejora la regresión lineal?

En la regresión lineal *sin regularización*, escalar **NO cambia las predicciones ni las métricas de error**. Solo cambia la escala de los coeficientes. Entonces, ¿por qué se hace?

1. **Para comparar la importancia de las features.** Con los datos escalados, cada coeficiente mide el efecto de 1 desviación típica, así que son comparables entre sí.
2. **Para los modelos con regularización (Ridge, Lasso, Elastic Net).** Ahí el escalado es casi obligatorio porque la penalización depende de la magnitud de los coeficientes.
3. **Para los modelos basados en Gradient Descent.** Si las escalas son muy distintas, el gradiente no apunta eficientemente al mínimo y tarda mucho en converger.

14. Validación cruzada (cross_val_score)

14.1 La limitación del train/test simple

Con un solo split train/test, los resultados dependen de qué muestras cayeron en cada grupo. Si tuviste suerte y las 20 muestras de test son fáciles, tus métricas saldrán brillantes. Si fueron difíciles, malas. No puedes fiarte del todo de un solo split.

14.2 La idea: dividir en K pliegues

La **validación cruzada** (cross-validation, CV) consiste en dividir los datos en K partes iguales (típicamente K = 5 o 10). Luego:

1. Entreno con 9 partes y evalúo con 1.
2. Repito cambiando qué parte queda fuera.
3. Al final tengo 10 métricas distintas, una por cada iteración.
4. Calculo la media y la desviación típica de esas 10 métricas.

La media te da una estimación más robusta del rendimiento del modelo. La desviación típica te dice lo variable que es según con qué datos entrenes: si es pequeña, tu modelo es estable; si es grande, tu modelo es sensible a los datos.

14.3 En sklearn

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(modelo, X, y,
                         cv=10,                          # 10 pliegues
                         scoring="neg_mean_absolute_error")

print("Scores por pliegue:", scores)
print("Media:", scores.mean())
print("Desviación típica:", scores.std())
```

¿POR QUÉ "NEG_MEAN_ABSOLUTE_ERROR" (NEGATIVO)?

Sklearn tiene la convención de que «más alto = mejor» para todos los scores. Pero el MAE es al revés: más bajo es mejor. Para resolverlo, sklearn le pone un signo negativo. Un MAE de 2.5 se reporta como −2.5. Tienes que tenerlo en cuenta al leer los resultados.

15. Condiciones para aplicar regresión lineal

La regresión lineal hace suposiciones sobre los datos. Si se violan gravemente, el modelo puede dar resultados engañosos (métricas buenas pero inferencia incorrecta). Estas condiciones son las que viste (o iba a ver) en las últimas diapositivas de la presentación de clase.

15.1 Linealidad entre target y predictores

La relación entre cada X y Y debe ser aproximadamente lineal. Se revisa con la matriz de correlaciones y scatter plots. Si los puntos forman una curva clara (una U, por ejemplo), la regresión lineal no va a funcionar bien y deberías plantearte una regresión polinómica (Parte IV).

15.2 No multicolinealidad

Los predictores deben ser razonablemente independientes entre sí. Si dos features correlacionan mucho (por ejemplo, $r > 0.8$), los coeficientes del modelo se vuelven inestables y difíciles de interpretar.

POR QUÉ LA MULTICOLINEALIDAD ES UN PROBLEMA

Imagina que tienes dos features casi idénticas (altura en metros y altura en centímetros). El modelo no sabe si el efecto sobre Y viene de una o de otra. Puede asignar coeficientes enormes y opuestos que se cancelan entre sí, volviendo el modelo inestable y sin sentido interpretativo.

Se detecta con la matriz de correlaciones entre features. Si dos features tienen correlación alta, se suele eliminar una de las dos (o combinarlas).

15.3 Homocedasticidad

Este palabro viene del griego y significa «igual varianza». La idea es que la **dispersión de los errores debe ser constante a lo largo del rango de las predicciones**.

- **Distribución homocedástica** (buena): los residuos tienen una dispersión parecida tanto para predicciones pequeñas como grandes.
- **Distribución heterocedástica** (mala): la dispersión cambia con la magnitud de la predicción. Por ejemplo, el modelo predice bien los precios bajos pero muy mal los altos.

Se detecta con un scatter plot de residuos vs predicciones. Si ves un patrón de «embudo» (más disperso a la derecha), tienes heterocedasticidad.

15.4 Independencia y normalidad de los residuos

- **Independencia:** los residuos no deben estar correlacionados entre sí. Si lo están (por ejemplo, en series temporales donde el error de un mes depende del anterior), la regresión lineal clásica no es apropiada.
- **Normalidad:** los residuos deben seguir aproximadamente una distribución normal. Se comprueba con un histograma: debe tener forma de campana centrada en 0.

15.5 Normalidad del target

No es estrictamente obligatorio, pero si el target tiene una distribución muy sesgada (por ejemplo, salarios con una cola larga a la derecha), el modelo puede funcionar peor. A veces conviene aplicar una transformación logarítmica al target antes de entrenar.

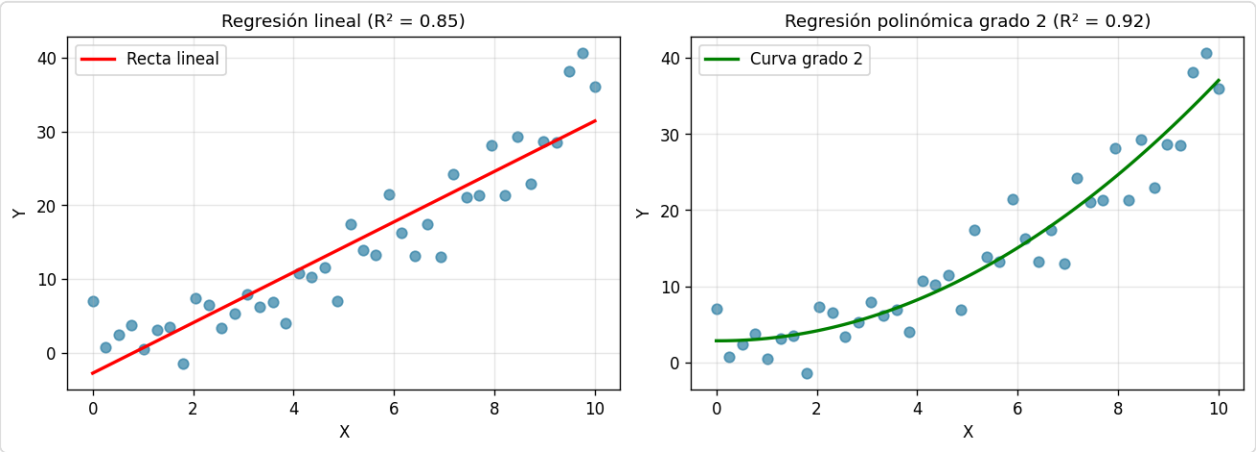
> **EN LA PRÁCTICA**
>
> En un análisis exploratorio típico para regresión lineal, siempre:
>
> 1. Dibujas el histograma del target para ver su distribución.
> 2. Dibujas la matriz de correlaciones con heatmap.
> 3. Dibujas scatter plots entre target y cada feature importante.
> 4. Después de entrenar, dibujas el histograma de residuos para comprobar normalidad.

Parte IV
Regresión polinómica

Hasta ahora hemos supuesto que la relación entre X e Y es una línea recta. Pero en la vida real muchas relaciones son curvas. La regresión polinómica es una extensión muy elegante de la regresión lineal que nos permite capturar esas curvas sin salirnos del mundo lineal.

16. Cuando la relación no es lineal

Supón que pintas una nube de puntos y ves claramente que siguen una forma curva: primero suben despacio, después más rápido, etc. Si intentas ajustar una recta, el modelo va a fallar sistemáticamente en las zonas curvas. La mejor recta posible aún será un mal modelo.



A la izquierda, una recta intentando ajustar puntos que siguen una curva: ajuste pobre (R^2 bajo). A la derecha, la misma nube con una curva polinómica de grado 2: se adapta a la forma real de los datos.

La idea genial de la regresión polinómica es: **añadir columnas nuevas al dataset que sean potencias de las originales**. Si X es la edad, creamos X^2, X^3, X^4... y las tratamos como si fueran features independientes. Después, aplicamos una regresión lineal clásica sobre ese dataset ampliado.

La ecuación que obtendremos tiene forma:

$$Y = \beta_0 + \beta_1 \cdot X + \beta_2 \cdot X^2 + \beta_3 \cdot X^3 + \ldots$$

POR QUÉ LA REGRESIÓN POLINÓMICA SIGUE SIENDO "LINEAL"

Aunque la curva resultante sea curvada, los parámetros β siguen apareciendo de forma lineal en la ecuación (multiplicados, no elevados). Por eso técnicamente sigue siendo una regresión *lineal* (lineal en los parámetros). Lo que cambiamos son las features, no el método de ajuste. Esto significa que podemos usar la misma `LinearRegression` de sklearn para entrenarla.

17. PolynomialFeatures de sklearn

Sklearn tiene una herramienta llamada **PolynomialFeatures** que automáticamente genera todas las potencias y combinaciones de las features originales hasta el grado que le pidas.

PolynomialFeatures(degree=3) crea columnas nuevas: x, x^2, x^3

x	x^2	x^3
2	4	8
3	9	27
5	25	125

PolynomialFeatures transforma cada valor de X en varias columnas: el propio X, X^2, X^3...

Si tienes **una sola feature** X y usas `PolynomialFeatures(degree=3)`, obtienes 3 columnas nuevas: X, X^2, X^3. Con una segunda feature Y añadida y degree=2, obtienes: X, Y, X^2, X·Y, Y^2. Es decir, no solo las potencias, también los **términos cruzados** (productos entre features). Esto es muy potente porque captura interacciones entre variables.

17.1 Uso típico en sklearn

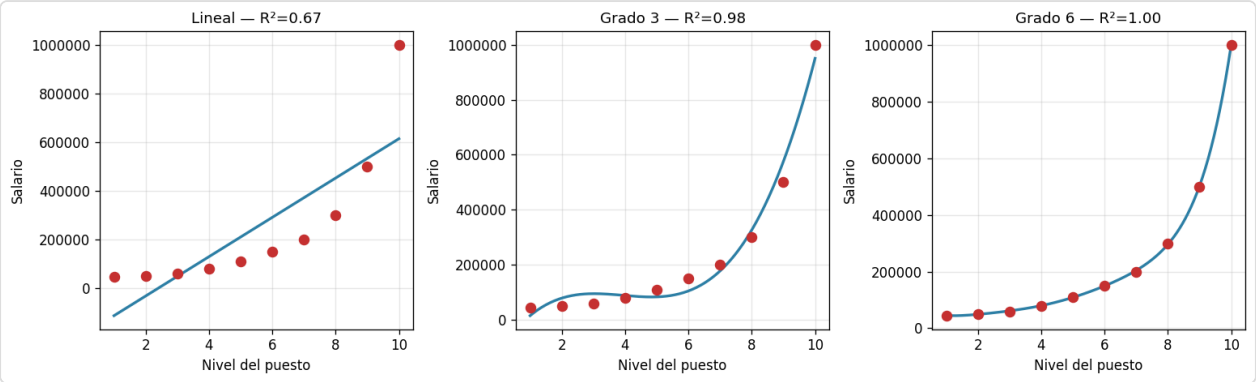
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Paso 1: crear el generador de features polinómicas
poly_feats = PolynomialFeatures(degree=2)

# Paso 2: ajustar con los datos de entrenamiento (aprende las columnas a
crear)
poly_feats.fit(X_train)

# Paso 3: transformar train y test en sus versiones polinómicas
X_train_poly = poly_feats.transform(X_train)
X_test_poly  = poly_feats.transform(X_test)

# Paso 4: entrenar una regresión lineal clásica sobre los datos ampliados
modelo = LinearRegression()
modelo.fit(X_train_poly, y_train)

# Paso 5: predecir (no olvides transformar también los datos nuevos)
y_pred = modelo.predict(X_test_poly)
```

ERROR TÍPICO: OLVIDAR TRANSFORMAR EL TEST

Una vez has entrenado con `X_train_poly`, el modelo espera features polinómicas siempre. Si luego le pasas `X_test` sin transformar, te dará un error de dimensiones o predicciones absurdas. Regla: si entrené con polinómica, predigo con polinómica.

17.2 El ejemplo del notebook de clase: salarios por nivel de puesto

En uno de los notebooks que vimos, el dataset *position_salaries* tenía una relación claramente exponencial: los puestos altos cobran *muchísimo* más que los bajos, no solo un poco más. La regresión lineal no podía capturar esa aceleración.



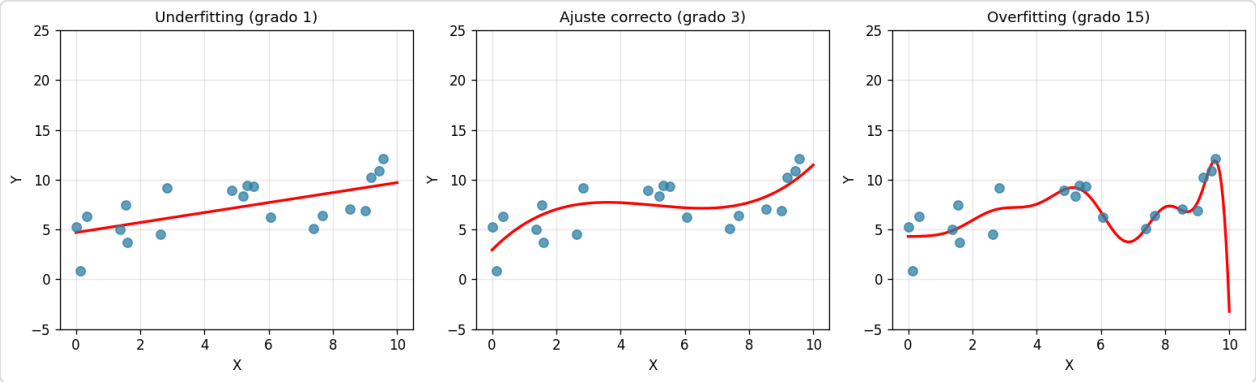
Dataset real de salarios por nivel. La regresión lineal (izquierda) falla porque la relación no es lineal. Grado 3 captura la forma general. Grado 6 se ajusta casi a la perfección.

Fíjate que el R^2 en train mejora mucho pasando de grado 1 a grado 6. Pero ojo: **mejorar el R^2 en train no siempre es bueno**. Aquí entra la advertencia del dilema sesgo-varianza que vimos en la Parte III.

18. Elección del grado óptimo

La pregunta natural es: ¿cuanto más alto el grado, mejor el modelo? **No.** Llegado un punto, el modelo se vuelve tan flexible que empieza a memorizar los datos de train, incluido el ruido. Eso es *overfitting*.

18.1 La imagen que tienes que grabarte

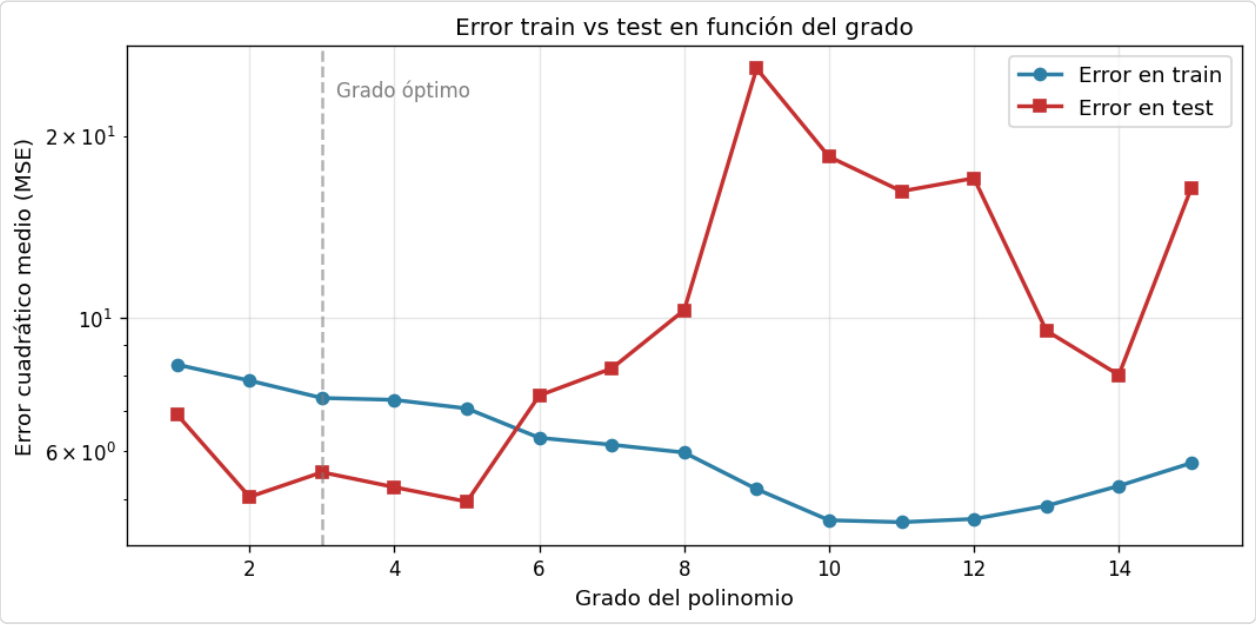


Tres modelos sobre los mismos datos. Grado 1 (izquierda): demasiado rígido, no captura la tendencia. Grado 3 (centro): sigue la forma general sin obsesionarse con cada punto. Grado 15 (derecha): la curva hace piruetas para pasar cerca de cada punto, incluida la parte del ruido aleatorio. Esto último es overfitting en estado puro.

Visualmente el grado 15 parece "ajustarse mejor" porque la curva está más cerca de cada punto. Pero fíjate en la parte derecha, cómo la curva cae en picado hacia abajo: eso es pura invención del modelo. Si apareciera un punto nuevo en X=10.5, grado 15 predeciría algo absurdo.

18.2 El gráfico diagnóstico: error en train vs error en test

La única forma honesta de elegir el grado es comparar el error en train con el error en test para distintos grados:



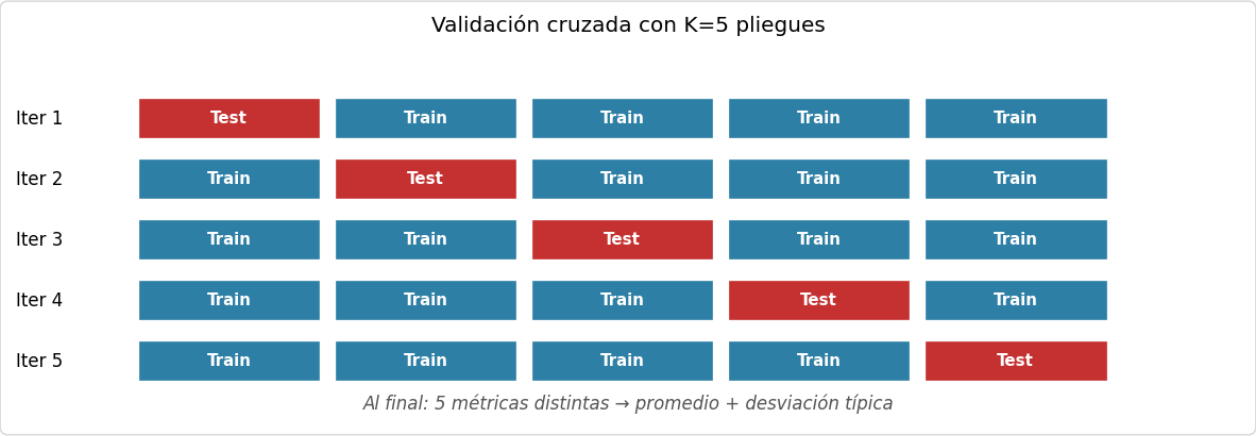
El error de train (azul) sigue bajando conforme el grado aumenta: el modelo cada vez se ajusta mejor a los datos que ha visto. Pero el error de test (rojo) baja al principio, toca un mínimo y después empieza a dispararse. Ese punto en el que el test toca su mínimo es el grado óptimo.

LA REGLA DE ORO

El grado óptimo es aquel que **minimiza el error en test**, no en train. Si el error en train es muy bajo pero el de test es alto, tienes overfitting: baja el grado. Si los dos errores son altos, tienes underfitting: sube el grado o añade más features.

18.3 Validación cruzada para elegir el grado

Con un solo split train/test la evaluación es ruidosa: depende de qué muestras cayeron en test. Para elegir el grado con más fiabilidad, usamos **validación cruzada** (que ya vimos en la Parte III). La idea recordatoria:



Con K=5 pliegues, entrenamos el modelo 5 veces. Cada vez, un pliegue distinto hace de test y los otros 4 de train. Al final tenemos 5 métricas: calculamos la media (estimación del rendimiento) y la desviación típica (estabilidad).

Aplicado al problema de elegir el grado:

```
from sklearn.model_selection import cross_val_score

for grado in [1, 2, 3, 4, 5, 6]:
    pf = PolynomialFeatures(degree=grado)
    X_poly = pf.fit_transform(X)
    scores = cross_val_score(LinearRegression(), X_poly, y,
                             cv=10, scoring="neg_mean_absolute_error")
    print(f"Grado {grado}: MAE medio = {-scores.mean():.2f}, std =
{scores.std():.2f}")
```

Elegimos el grado que dé mejor MAE medio con desviación típica razonable. Si dos grados empatan en MAE medio, elige el de menor desviación típica (más estable) y el menor grado posible (principio de parsimonia: a igualdad de rendimiento, modelos simples son preferibles).

18.4 Resumen operacional de regresión polinómica

Paso	Qué hacer
1	Visualizar X vs Y con scatter plot para detectar si la relación es curva
2	Partir los datos en train/test antes de nada
3	Probar varios grados (1, 2, 3, 4, 5, 6) y comparar error train vs test
4	Elegir el grado que minimice error en test
5	(Recomendado) Confirmar la elección con validación cruzada
6	Cuidado con grados muy altos: el overfitting explota rápido

PARA RETENER

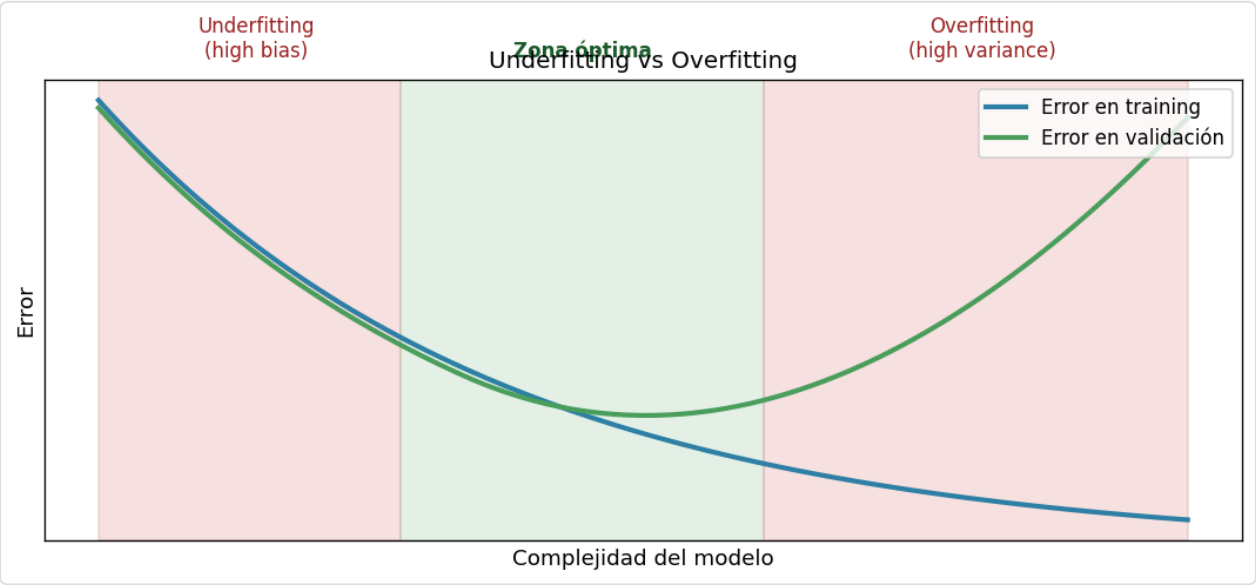
La regresión polinómica **no es un modelo distinto** de la lineal. Es la misma regresión lineal aplicada a features transformadas. Todo lo que aprendimos sobre coeficientes, R^2, MAE, train/test se aplica exactamente igual. El único hiperparámetro nuevo es el *grado*, y se elige con el mismo diagnóstico que cualquier hiperparámetro: probando varios valores y comparando el rendimiento en test.

Parte V
Regularización: Ridge, Lasso y Elastic Net

En la Parte III vimos el problema del overfitting. En la Parte IV vimos cómo la polinómica se va rápidamente a overfitting si subimos el grado. La regularización es la técnica estándar para controlar el overfitting en regresión lineal (y también en logística). Una vez entiendas Ridge, el resto (Lasso, Elastic Net) son variaciones sobre la misma idea.

19. Por qué regularizar

19.1 Recordatorio: el problema del overfitting



Cuando el modelo es demasiado simple (izquierda), ambos errores son altos: underfitting. Cuando es demasiado complejo (derecha), el error de train sigue bajando pero el de validación se dispara: overfitting. La zona central es la óptima. La regularización es una herramienta para empujar hacia esa zona.

19.2 La idea de la regularización en una frase

QUÉ HACE LA REGULARIZACIÓN

En lugar de dejar que el modelo minimice *solo* el error, le añadimos una penalización extra proporcional al **tamaño de los coeficientes β**. El modelo ahora tiene que elegir: o reduzco el error, o reduzco los coeficientes, pero no puedo tener ambos exageradamente altos.

El resultado: coeficientes más pequeños → modelo menos agresivo → menos overfitting → generaliza mejor.

La función de costes original (regresión lineal) era:

$$\mathrm{MSE} = (1/\mathrm{N}) \cdot \Sigma\ (\mathrm{Y}_i - \hat{\mathrm{Y}}_i)^2$$

Con regularización, añadimos un término de penalización λ · Reg(β):

$$\mathrm{J}(\beta) = \mathrm{MSE} + \lambda \cdot \mathrm{Reg}(\beta)$$

Donde **λ** (lambda) es el hiperparámetro que controla *cuánto* regularizamos. En sklearn este parámetro se llama `alpha`. Lo que cambia según la técnica es la forma de *Reg(β)*:

- **Ridge (L2):** Reg(β) = Σ β^2 — penaliza coeficientes grandes al cuadrado.
- **Lasso (L1):** Reg(β) = Σ |β| — penaliza el valor absoluto de los coeficientes.
- **Elastic Net:** mezcla de L1 y L2.

EL HIPERPARÁMETRO ALPHA EN SKLEARN

- **alpha = 0** → sin penalización → equivale a regresión lineal normal.
- **alpha bajo** (ej. 0.001) → regularización suave, coeficientes casi intactos.
- **alpha alto** (ej. 100) → regularización fuerte, coeficientes muy reducidos, modelo casi plano.
- **alpha** → ∞ → todos los coeficientes cercanos a 0 → el modelo predice una constante.

El valor óptimo se busca probando varios alphas y comparando el error en validación (o mejor, con validación cruzada).

ANTES DE REGULARIZAR: ESTANDARIZA

La regularización penaliza los coeficientes grandes. Pero si tienes features en escalas muy distintas (edad 20-50 vs ingresos 20.000-100.000), los coeficientes ya parten siendo muy distintos solo por la escala. La regularización castigaría injustamente a las features en unidades pequeñas.

Regla sagrada: aplica siempre `StandardScaler` antes de usar Ridge, Lasso o Elastic Net. Es prácticamente obligatorio.

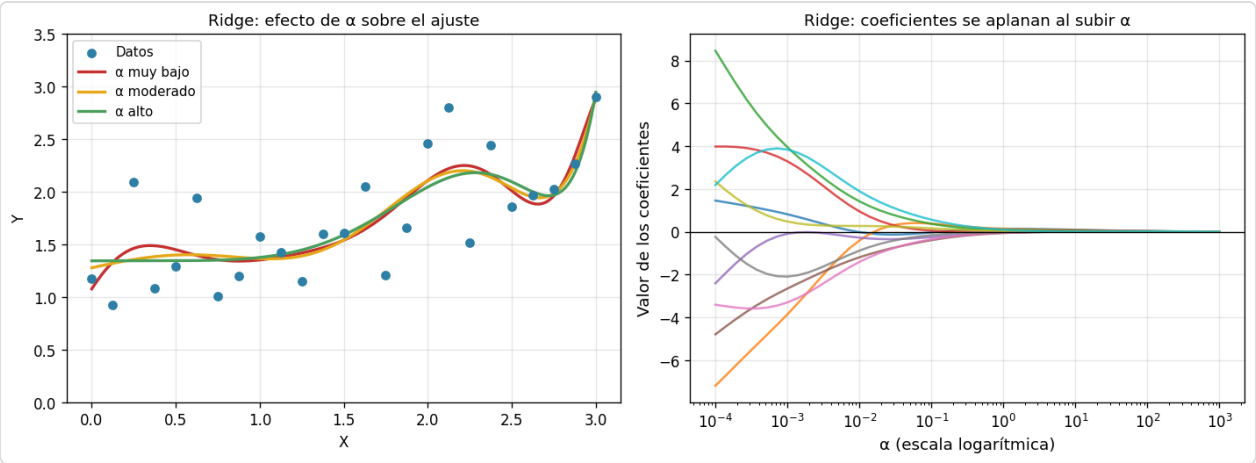
20. Ridge (L2)

20.1 La fórmula

$$J(\beta) = \Sigma\ (Y_i - \hat{Y}_i)^2 + \lambda \cdot \Sigma\ \beta_j^2$$

Leído en cristiano: minimiza el error cuadrático habitual, pero no dejes que la suma de los coeficientes al cuadrado sea grande.

20.2 Qué efecto tiene en los coeficientes



Izquierda: efecto visual de alpha sobre un ajuste polinómico. Con alpha muy bajo el modelo se retuerce (overfitting). Con alpha alto la curva se aplana. Derecha: cómo los coeficientes van reduciéndose progresivamente hacia 0 conforme aumenta alpha — pero ***nunca llegan exactamente a cero****.*

LO CARACTERÍSTICO DE RIDGE

Ridge **reduce** todos los coeficientes, pero conserva todas las features. No es capaz de poner un coeficiente exactamente a cero. El modelo final usa todos los predictores, solo que con pesos más moderados.

Esto hace a Ridge especialmente bueno cuando *todas* o *la mayoría* de features contribuyen algo al resultado y no quieres descartar ninguna.

20.3 Uso en sklearn

```
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler

# Paso 1: escalar (obligatorio)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s  = scaler.transform(X_test)

# Paso 2: entrenar Ridge con algún alpha
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_s, y_train)

# Paso 3: predecir
predicciones = ridge.predict(X_test_s)
```

20.4 Cómo encontrar el alpha óptimo

La forma típica es probar muchos alphas en escala logarítmica y quedarnos con el que minimice el error en validación:

```
import numpy as np

alphas = np.logspace(-4, 3, 100)  # 100 valores entre 0.0001 y 1000
errores = []
for a in alphas:
    m = Ridge(alpha=a)
    m.fit(X_train_s, y_train)
    pred = m.predict(X_test_s)
    errores.append(mean_squared_error(y_test, pred))

alpha_optimo = alphas[np.argmin(errores)]
print(f"Alpha óptimo: {alpha_optimo}")
```

Este patrón aparece en los notebooks de clase de Prostate Cancer y Advertising. Lo conocido es que se usa `np.logspace` (escala logarítmica) porque alpha puede variar en órdenes de magnitud enormes.

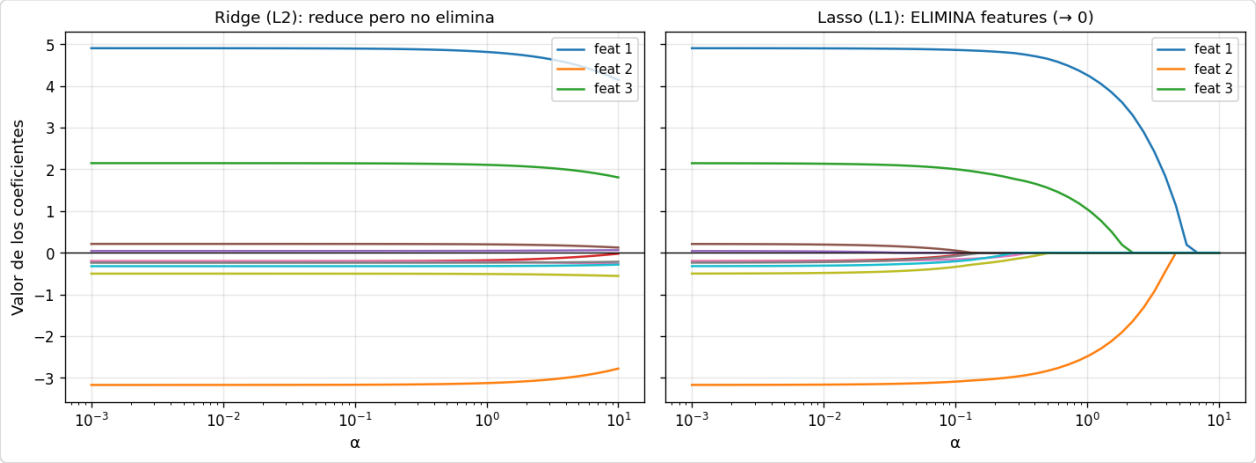
21. Lasso (L1)

21.1 La fórmula

$$J(\beta) = \Sigma\ (Y_i - \hat{Y}_i)^2 + \lambda \cdot \Sigma\ |\beta_j|$$

El cambio parece pequeño: en lugar de β^2 usamos $|\beta|$ (valor absoluto). Pero matemáticamente esto tiene una consecuencia enorme.

21.2 La propiedad mágica: feature selection



Mismo dataset, mismos datos. Izquierda: Ridge reduce los coeficientes pero ninguno llega exactamente a cero. Derecha: Lasso sí empuja coeficientes a cero exactos conforme sube alpha. A la derecha, las features 4, 5, 6... (ruido en este dataset) desaparecen completamente.

LASSO = FEATURE SELECTION AUTOMÁTICA

A diferencia de Ridge, Lasso **sí es capaz de poner coeficientes exactamente a cero**. Esto significa que, de facto, *elimina features del modelo*. Las features que sobreviven son las que el modelo considera relevantes.

Por eso Lasso se conoce como un método de *feature selection*: no solo mejora la generalización, también te dice qué features son importantes.

21.3 Cuándo usar Lasso

- Cuando **sospechas que solo unas pocas features son relevantes** y el resto es ruido.
- Cuando tienes **muchísimas features** (cientos o miles) y quieres un modelo interpretable con las más importantes.
- Cuando quieres un modelo **"sparse"** (con pocos coeficientes no nulos).

21.4 Punto débil

Cuando dos features están muy correlacionadas entre sí, Lasso se comporta de forma errática: elige una arbitrariamente y pone la otra a cero. Si reentreansacas con datos ligeramente distintos, puede elegir la otra. Esto se resuelve con Elastic Net.

21.5 Uso en sklearn

```
from sklearn.linear_model import Lasso

lasso = Lasso(alpha=0.1, max_iter=10000)
lasso.fit(X_train_s, y_train)

# Las features con coeficiente != 0 son las seleccionadas
import numpy as np
features_importantes = np.where(lasso.coef_ != 0)[0]
print("Features seleccionadas:", features_importantes)
```

`max_iter=10000`: Lasso se resuelve iterativamente y con el valor por defecto (1000) puede avisarte de que no ha convergido. Subirlo resuelve el warning.

22. Elastic Net

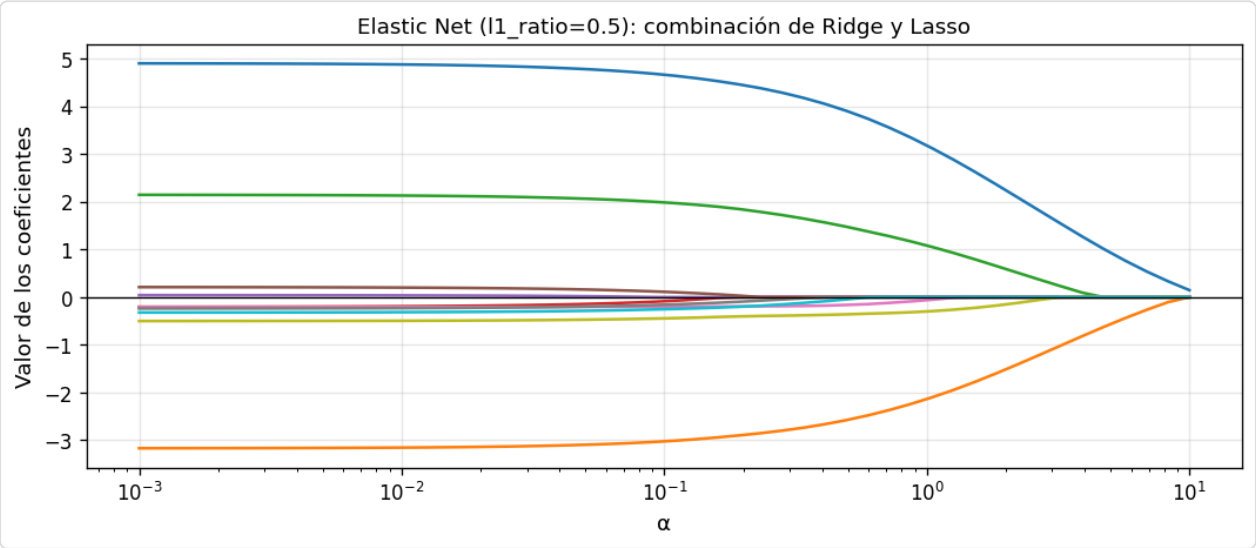
22.1 Lo mejor de los dos mundos

Elastic Net combina las penalizaciones L1 y L2 en una sola fórmula:

$$J(\beta) = \text{MSE} + \lambda \cdot [\ r \cdot \Sigma\ |\beta| + (1 - r) \cdot \Sigma\ \beta^2\ ]$$

Donde `r` es el parámetro **l1_ratio** en sklearn y controla la mezcla:

- `l1_ratio = 0` → solo L2 → equivale a Ridge puro.
- `l1_ratio = 1` → solo L1 → equivale a Lasso puro.
- `l1_ratio = 0.5` → mitad y mitad (compromiso).



Elastic Net con l1_ratio=0.5: se comporta de forma intermedia entre Ridge y Lasso. Algunos coeficientes llegan a cero (como en Lasso) pero la transición es más suave y agrupa mejor las features correlacionadas.

22.2 Cuándo usar Elastic Net

- Cuando tienes **muchas features correlacionadas entre sí**. Lasso sufriría; Elastic Net las agrupa y reparte pesos entre ellas.
- Cuando quieres feature selection (como Lasso) pero necesitas más estabilidad que la de Lasso puro.
- Cuando quieres el beneficio de Ridge (estabilidad con features correlacionadas) pero también quieres eliminar las irrelevantes.

22.3 Uso en sklearn

```
from sklearn.linear_model import ElasticNet

en = ElasticNet(alpha=0.1, l1_ratio=0.5, max_iter=10000)
en.fit(X_train_s, y_train)
```

23. Cómo elegir entre Ridge, Lasso y Elastic Net

Comparativa Ridge vs Lasso vs Elastic Net

Característica	Ridge (L2)	Lasso (L1)	Elastic Net
Penalización	$\Sigma \beta^2$	$\Sigma \lvert\beta\rvert$	Mezcla L1 + L2
Reduce coeficientes	Sí	Sí	Sí
Pone coef. a cero	No	Sí (feature selection)	Sí
Con features correladas	Comparte peso	Elige una arbitrariamente	Agrupa (mejor)
Hiperparámetros	alpha	alpha	alpha + l1_ratio

Resumen comparativo de las tres técnicas. Cada una tiene su nicho de aplicación óptimo.

23.1 Guía práctica de decisión

Escenario	Técnica recomendada
Regresión sin problemas obvios → tirando a defecto	**Ridge** (buena opción por defecto, poco riesgo)
Sospechas que pocas features son relevantes	**Lasso** (hace feature selection)
Muchas features, y muchas correlacionadas entre sí	**Elastic Net** (maneja correlación + selecciona)
Hay decenas o cientos de miles de features	**Lasso o Elastic Net** (reducen dimensionalidad)
Quieres un modelo simple e interpretable	**Lasso** (solo sobreviven las importantes)

CONSEJO PRÁCTICO

En proyectos reales, lo habitual es probar los tres, elegir el alpha óptimo con validación cruzada para cada uno, y quedarse con el que mejor rendimiento tenga en test. Sklearn tiene versiones automáticas: `RidgeCV`, `LassoCV` y `ElasticNetCV` que buscan el alpha óptimo por ti.

```
from sklearn.linear_model import RidgeCV, LassoCV, ElasticNetCV

# Prueban muchos alphas internamente y eligen el mejor con CV
ridge_cv = RidgeCV(alphas=np.logspace(-4, 3, 100))
ridge_cv.fit(X_train_s, y_train)
print("Mejor alpha Ridge:", ridge_cv.alpha_)
```

23.2 La regla de oro que cierra la Parte V

SIEMPRE QUE HAGAS REGRESIÓN, PIENSA EN REGULARIZACIÓN

La tentación es empezar siempre con `LinearRegression()`. Pero en la práctica, añadir una pizca de Ridge (alpha bajo, por ejemplo 0.1) casi nunca empeora el modelo y a menudo lo mejora. Una cita típica del mundo ML: *"toda regresión se beneficia de algo de regularización"*.

Parte VI
Clasificación y regresión logística

*Hasta ahora hemos predicho variables continuas (precios, notas, salarios). Pero muchos problemas reales consisten en predecir una **categoría**: spam o no spam, clic o no clic, sano o enfermo. Eso es clasificación. Y el algoritmo de partida para clasificar, por mucho que su nombre despiste, se llama regresión logística.*

24. Clasificación: la otra rama del aprendizaje supervisado

Recordamos: dentro del aprendizaje supervisado hay dos tipos de problemas:

- **Regresión:** el target es un número continuo (precio, temperatura, nota).
- **Clasificación:** el target es una categoría (spam/no spam, tipo de flor, dígito del 0 al 9).

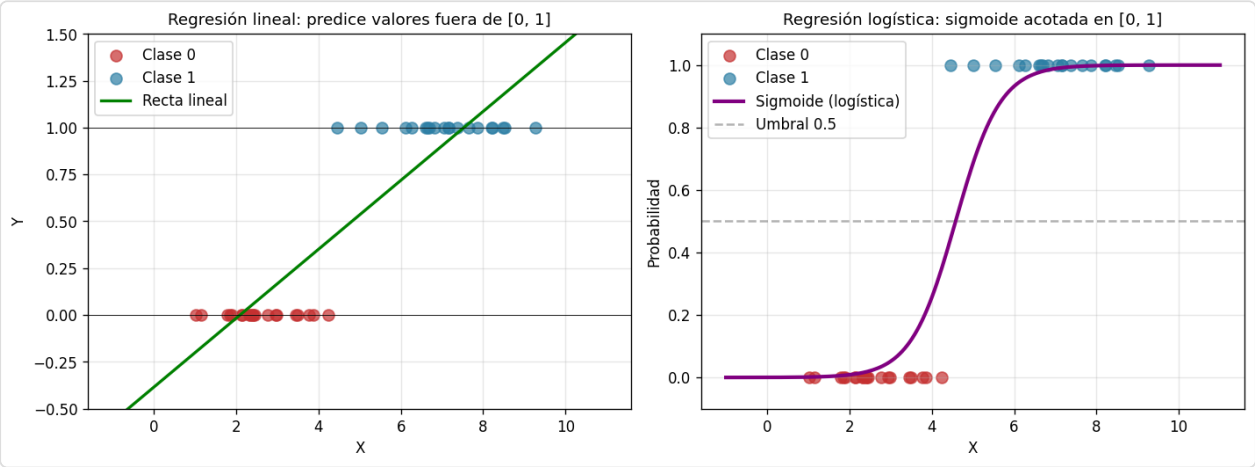
Dentro de clasificación, se distinguen dos variantes:

- **Clasificación binaria:** solo hay dos clases. Ejemplos: spam/no spam, paciente enfermo/sano, cliente impaga/no impaga. Típicamente se codifica con 0 y 1, donde 1 es la clase que nos interesa detectar.
- **Clasificación multi-clase:** hay más de dos clases. Ejemplos: identificar dígitos del 0 al 9, clasificar una flor en setosa/versicolor/virginica, determinar el tipo de vino.

Los algoritmos de clasificación más habituales son: **regresión logística**, árboles de decisión, k-NN, Naive Bayes, SVM, random forest y redes neuronales. En esta guía nos centramos en la logística, que es la que has visto en clase.

25. Por qué la regresión lineal no sirve para clasificar

Es una pregunta razonable: si tenemos un problema binario (0 o 1), ¿por qué no usar directamente una regresión lineal prediciendo 0 y 1 como si fueran números?



Izquierda: una regresión lineal ajustada a datos binarios produce predicciones fuera del rango [0, 1], incluyendo valores negativos o mayores que 1, que no tienen sentido como probabilidades. Derecha: la regresión logística usa la función sigmoide para acotar siempre la predicción entre 0 y 1, interpretable como una probabilidad.

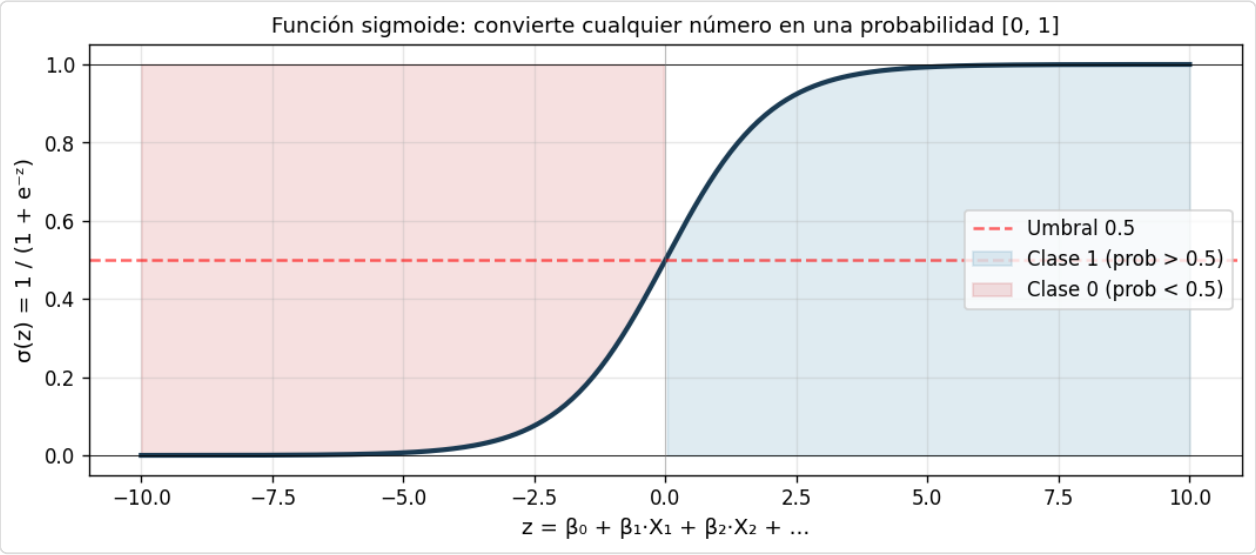
Los problemas de usar lineal para clasificar son:

- Las predicciones no están acotadas: puedes obtener 1.5 o −0.3, que no significan nada como probabilidad.
- La recta es muy sensible a los valores extremos y distorsiona la frontera de decisión.
- Asume que la relación entre X y P(Y=1) es lineal, cuando en realidad suele ser *sigmoide* (crece despacio, después rápido en el medio, después se estabiliza).

26. La función sigmoide

La regresión logística resuelve el problema aplicando una función matemática que transforma cualquier número del universo en un valor entre 0 y 1. Esa función se llama **sigmoide**:

$$\sigma(z) = 1 \ / \ (1 + e^{-z})$$



La función sigmoide toma cualquier número (eje X) y devuelve siempre un valor entre 0 y 1 (eje Y). En z=0 vale exactamente 0.5, el punto de decisión. Para z grandes positivos tiende a 1 (clase 1 muy probable); para z muy negativos tiende a 0 (clase 0 muy probable).

La regresión logística funciona así:

1. Calcula una combinación lineal de las features, exactamente como en la regresión lineal: $z = \beta_0 + \beta_1 \cdot X_1 + \beta_2 \cdot X_2 + \ldots$
2. Pasa ese valor z por la sigmoide para obtener una probabilidad entre 0 y 1.
3. Compara la probabilidad con un **umbral** (por defecto 0.5) para decidir la clase: si prob > 0.5, predice 1; si no, predice 0.

> **PARA RETENER**
>
> La regresión logística es, en esencia, una regresión lineal con una sigmoide encima que transforma el output en una probabilidad. Todo lo que aprendimos sobre coeficientes β, gradient descent, regularización, train/test sigue aplicando con pequeñas adaptaciones.

27. Formulación matemática

A continuación recorremos, paso a paso, la deducción que vio tu clase. No hace falta que te la aprendas de memoria; el objetivo es entender de dónde sale cada cosa.

27.1 Paso 1: partimos de la sigmoide

$$\sigma(y) = 1 \,/\, (1 + e^{-y})$$

27.2 Paso 2: aplicamos la sigmoide a la combinación lineal

La probabilidad de que Y=1 dado X es:

$$P(Y=1 \mid X) = 1 \,/\, (1 + e^{-(\beta_0 + \beta_1 X_1 + \ldots + \beta_p X_p)})$$

27.3 Paso 3: probabilidad de que Y=0

Como solo hay dos clases y las probabilidades suman 1:

$$P(Y=0 \mid X) = 1 - P(Y=1 \mid X)$$

27.4 Paso 4: log-odds (lineal de nuevo)

Si dividimos las dos probabilidades y aplicamos logaritmo natural, se simplifica enormemente y obtenemos una ecuación lineal limpia:

$$\ln[\, P(Y=1) \,/\, P(Y=0) \,] = \beta_0 + \beta_1 \cdot X_1 + \ldots + \beta_p \cdot X_p$$

El término de la izquierda se llama **log-odds** o *logit*: es el logaritmo del cociente entre la probabilidad de que ocurra el evento y la probabilidad de que no ocurra.

> **LA INTERPRETACIÓN DE LOS COEFICIENTES EN LOGÍSTICA**
>
> En regresión lineal, β_j era "cuánto sube Y por cada unidad de X_j". En logística, β_j es "cuánto sube el *log-odds* por cada unidad de X_j". Es menos intuitivo, pero para *feature importance* funciona igual: estandariza las features y compara los β en valor absoluto.

28. Gradient Descent para logística

Igual que en regresión lineal, la regresión logística tiene su propia **función de costes** que hay que minimizar. No es el MSE (ese solo funciona bien con respuestas continuas). En logística se usa la *log-loss* o *cross-entropy*:

$$J(\beta) = -(1/m) \cdot \Sigma \,[\, y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i) \,]$$

Esta función tiene dos partes que se activan según la clase real del ejemplo:

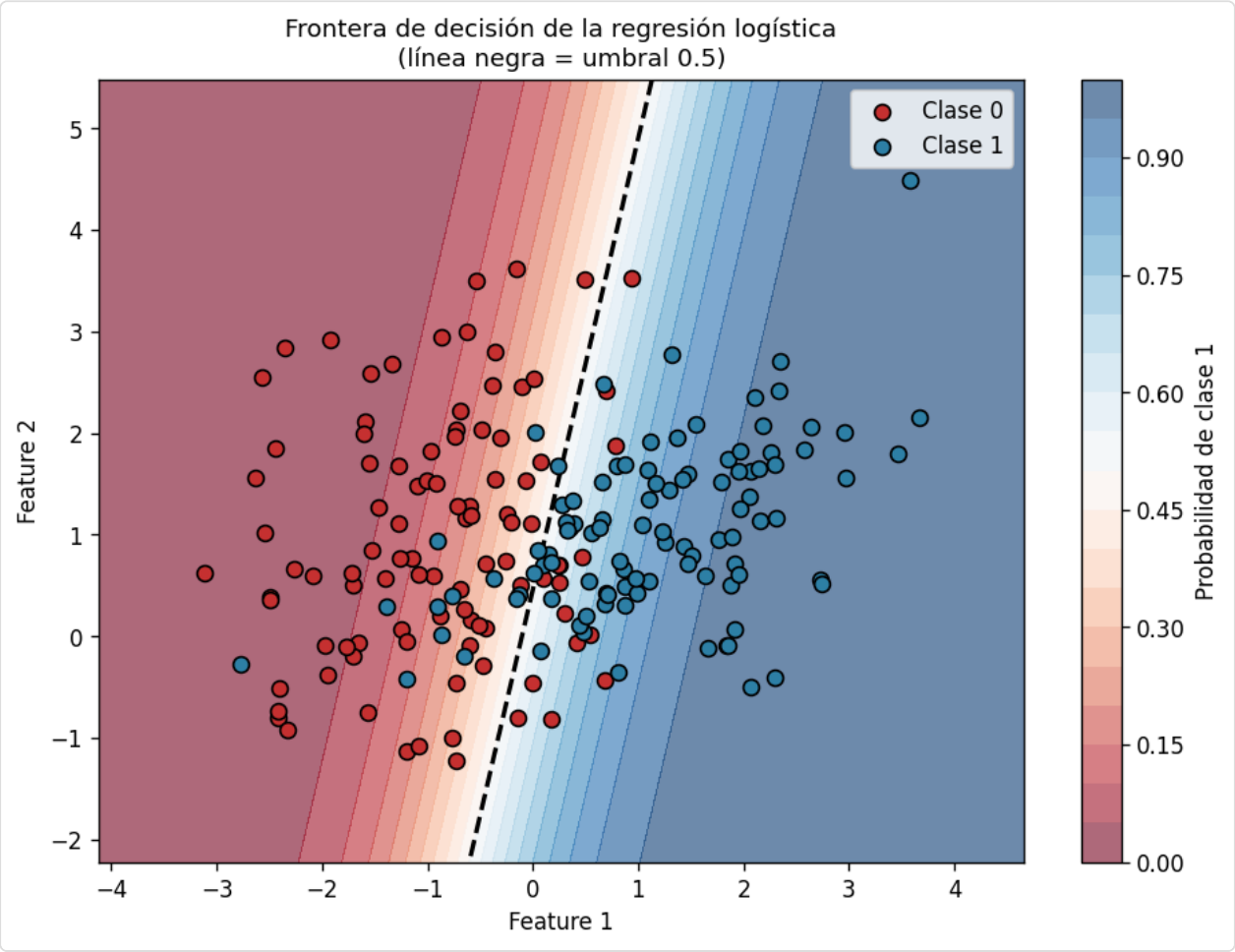
- Si la clase real es $y=1$: el segundo término desaparece, queda $\log(\hat{y})$. Si $\hat{y}$ se acerca a 1, $\log(\hat{y})$ se acerca a 0 (poco castigo). Si $\hat{y}$ se acerca a 0 (modelo muy equivocado), $\log(\hat{y})$ se dispara a $-\infty$ (mucho castigo).
- Si la clase real es $y=0$: el primer término desaparece, queda $\log(1-\hat{y})$. Análogo: castiga mucho predecir 1 cuando era 0.

Para encontrar los β que minimizan esta función se usa el mismo algoritmo que ya vimos en la Parte I: **Gradient Descent**. El código en sklearn es casi idéntico al de la regresión lineal:

```
from sklearn.linear_model import LogisticRegression

modelo = LogisticRegression(max_iter=1000)
modelo.fit(X_train, y_train)
predicciones = modelo.predict(X_test)
```

29. Frontera de decisión

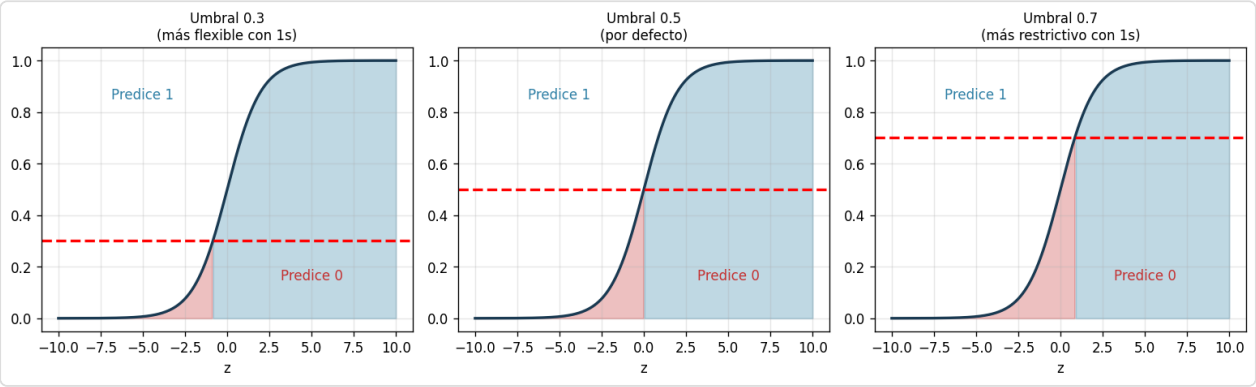
La regresión logística traza una **frontera de decisión** en el espacio de features: todos los puntos por encima de la frontera se clasifican como 1, todos los de debajo como 0. Con dos features, la frontera es una recta (o curva si hay términos polinómicos); con más, un hiperplano.



Dos clases en un espacio 2D. La regresión logística encuentra la línea que mejor separa ambas clases. El color de fondo muestra la probabilidad de pertenecer a la clase 1: cuanto más rojo, más probable; cuanto más azul, menos probable. La línea negra discontinua es la frontera del umbral 0.5.

30. El umbral (threshold) y por qué importa

Por defecto, sklearn aplica un umbral de 0.5: si la probabilidad ≥ 0.5, predice 1. Pero **este umbral es modificable** y cambiarlo puede ser crucial en problemas reales.



El mismo modelo con tres umbrales distintos. Izquierda (umbral 0.3): flexible con los 1s, más puntos se clasifican como 1. Centro (umbral 0.5): el habitual. Derecha (umbral 0.7): restrictivo, solo los casos muy seguros se clasifican como 1.

CÓMO AFECTA MODIFICAR EL UMBRAL

- **Umbral más bajo (ej. 0.3):** el modelo es más flexible con los 1s → más *falsos positivos* (FP suben), menos *falsos negativos* (FN bajan) → recall sube.
- **Umbral más alto (ej. 0.7):** el modelo es más restrictivo con los 1s → menos FP, más FN → precision sube.

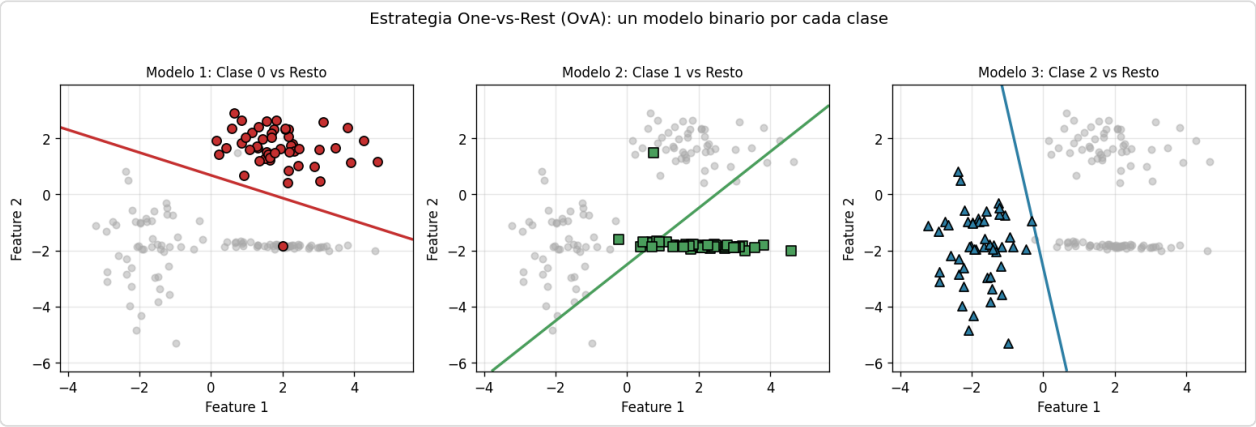
La elección del umbral depende del problema: en detección de ladrones no podemos permitirnos dejar escapar ni uno (umbral bajo, recall alto); en clasificación de videos infantiles no podemos dejar colar ningún contenido malo (umbral alto, precision alta). Lo veremos en detalle en la Parte VII.

Para modificar el umbral en sklearn no usas `predict` sino `predict_proba` y aplicas tu propia regla:

```
# Con umbral por defecto (0.5)
predicciones = modelo.predict(X_test)

# Con umbral personalizado (ej. 0.3)
probabilidades = modelo.predict_proba(X_test)[:, 1]  # prob de clase 1
predicciones_umbral = (probabilidades >= 0.3).astype(int)
```

31. Multiclase: estrategia One-vs-Rest

La regresión logística es esencialmente binaria (dos clases). Pero tu clase también vio el ejemplo de los dígitos del 0 al 9, que son 10 clases. ¿Cómo lo hace? Con la estrategia **One-vs-Rest** (también llamada One-vs-All).



Con 3 clases, One-vs-Rest entrena 3 modelos binarios. Cada uno aprende a distinguir una clase concreta del resto. Al predecir un punto nuevo, se consultan los 3 modelos y gana el que dé la probabilidad más alta.

Para el problema de los dígitos (10 clases):

1. Se entrenan 10 modelos binarios: "¿es un 0 o no?", "¿es un 1 o no?", ..., "¿es un 9 o no?".
2. Al predecir un dígito nuevo, los 10 modelos dan 10 probabilidades.
3. La clase predicha es la del modelo con mayor probabilidad.

Sklearn hace todo esto automáticamente: cuando entrenas `LogisticRegression` con un target de más de dos clases, internamente usa OvR sin que tengas que configurar nada.

32. Condiciones para aplicar regresión logística

Son más relajadas que las de la regresión lineal:

- **No colinealidad** entre features (igual que en lineal).
- **Independencia de las observaciones** (crítico si son series temporales).
- **Muestras grandes funcionan mejor.** La logística necesita al menos unas decenas de ejemplos por clase.
- **Ausencia de outliers.**
- **NO necesita que la relación X-Y sea lineal**, porque la sigmoide ya es no lineal.
- **NO necesita que los errores se distribuyan normalmente.**

Parte VII
Métricas de clasificación

Esta es la parte que me dijiste que no entendías nada. Vamos a construirlo todo desde cero. Spoiler: una vez entiendas la matriz de confusión, todas las demás métricas salen de ella como combinaciones aritméticas. No hay magia.

---

33. Accuracy: la métrica más obvia (y traicionera)

33.1 Qué es

La métrica más simple imaginable: el porcentaje de aciertos sobre el total.

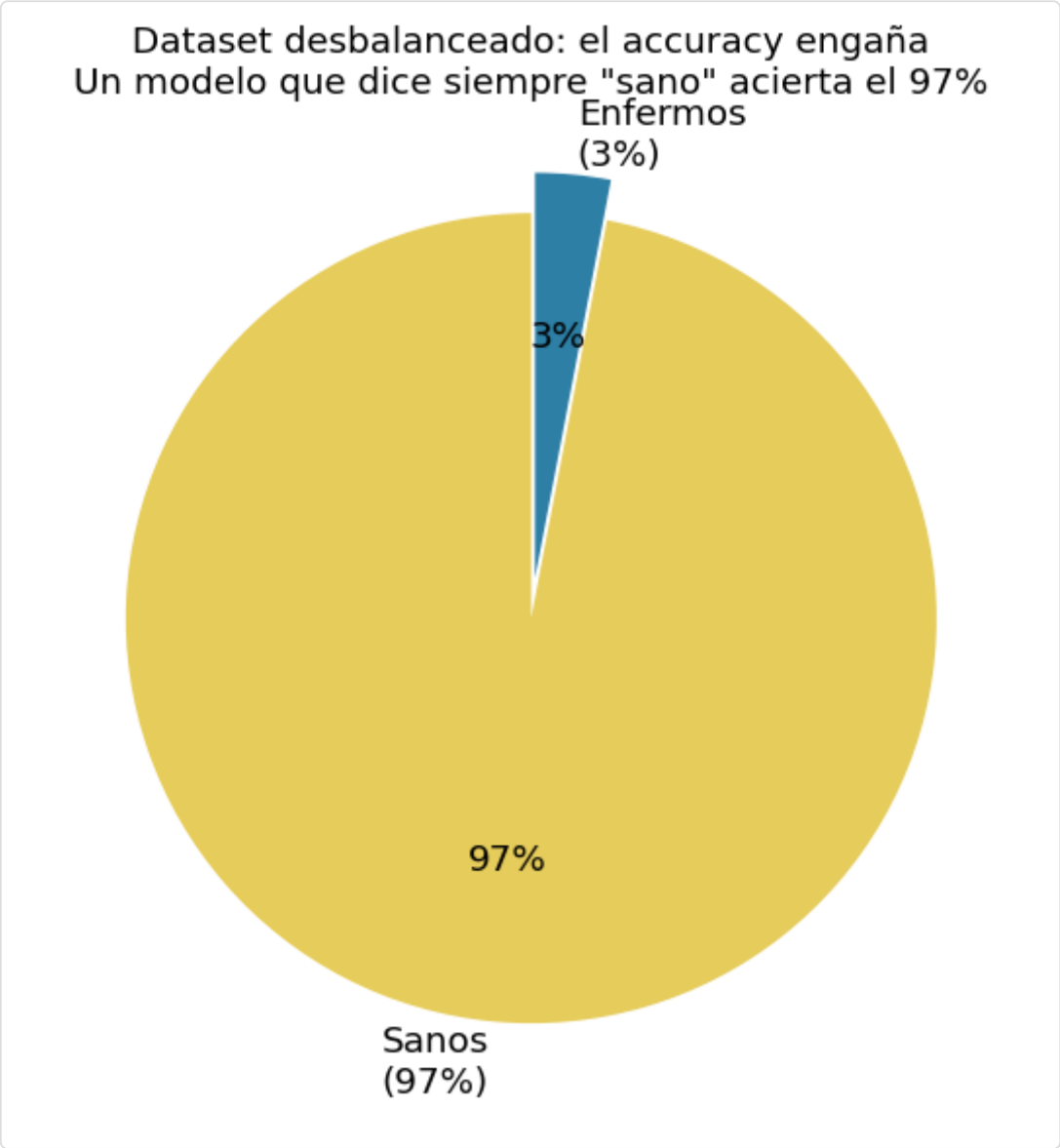
$$\text{Accuracy} = \text{aciertos} \ / \ \text{total predicciones}$$

EJEMPLO

Un modelo predice sobre 100 imágenes. Acierta 85. Accuracy = 85/100 = 0.85 = 85%.

33.2 Por qué engaña en datasets desbalanceados

Imagínate que tienes un dataset médico donde el 97% de los pacientes están sanos y solo el 3% tienen cierta enfermedad.



Dataset extremadamente desbalanceado: por cada paciente enfermo hay 32 sanos. Un modelo tonto que siempre predijera "sano" acertaría en el 97% de los casos.

EL ACCURACY MIENTE CON CLASES DESBALANCEADAS

Un modelo que siempre predice "sano" tendría un accuracy del 97%. Parece un gran modelo, ¿verdad? Pero **no detecta a ningún paciente enfermo**, que es precisamente lo único que nos importa. El accuracy es útil solo cuando las clases están razonablemente balanceadas.

Posibles soluciones al desbalanceo:

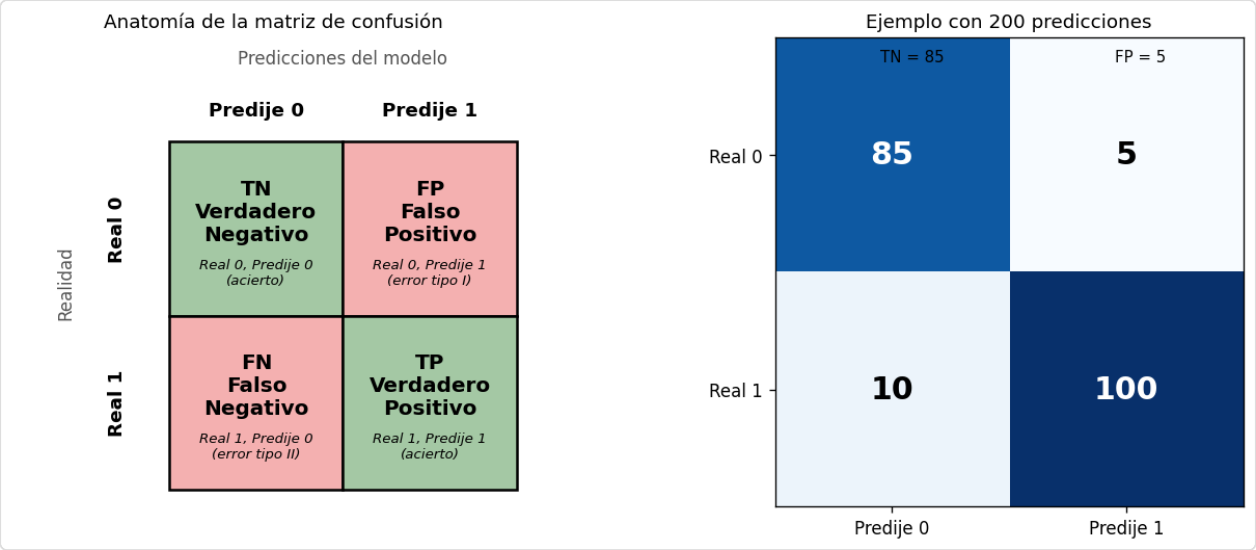
1. **Cambiar la métrica:** usar precision, recall, F1, AUC (lo veremos a continuación).
2. **Conseguir más datos de la clase minoritaria.**
3. **Resampling:** duplicar ejemplos de la clase minoritaria (*oversampling*) o eliminar ejemplos de la mayoritaria (*undersampling*).

4. **Generar datos sintéticos** con técnicas como SMOTE.

34. Matriz de confusión

La matriz de confusión es la herramienta básica para entender qué está haciendo realmente tu modelo, más allá del porcentaje global. Es una tabla 2x2 (para clasificación binaria) que cuenta los aciertos y los errores de cada tipo.

34.1 Anatomía de la matriz



Izquierda: las cuatro celdas de la matriz de confusión con su significado. Derecha: un ejemplo real con 200 predicciones. Las celdas de la diagonal (TN y TP, en azul fuerte) son los aciertos; las celdas fuera de la diagonal (FP y FN) son los errores.

Las cuatro casillas:

Sigla	Nombre	Significado
TP	True Positive (Verdadero Positivo)	La realidad era 1, el modelo predijo 1. **Acierto.**
TN	True Negative (Verdadero Negativo)	La realidad era 0, el modelo predijo 0. **Acierto.**
FP	False Positive (Falso Positivo)	La realidad era 0, el modelo predijo 1. **Error tipo I.** "Falsa alarma".
FN	False Negative (Falso Negativo)	La realidad era 1, el modelo predijo 0. **Error tipo II.** "Se me ha escapado".

REGLA MNEMOTÉCNICA

La primera letra (T o F) indica si el modelo acertó (**T**rue) o falló (**F**alse). La segunda (P o N) indica qué predijo el modelo (**P**ositive o **N**egative). Por tanto:

- **TP** = "acerté prediciendo positivo"
- **FN** = "fallé prediciendo negativo" (es decir, debería haber dicho positivo)

34.2 Por qué importa qué es la clase 1

La convención es que la **clase 1 es la que nos interesa detectar**: el paciente enfermo, el ladrón, el cliente que impaga, el email spam, el clic en el anuncio. La clase 0 es la normal/negativa. Esto es crítico porque todas las métricas que veremos dependen de esa convención.

34.3 En sklearn

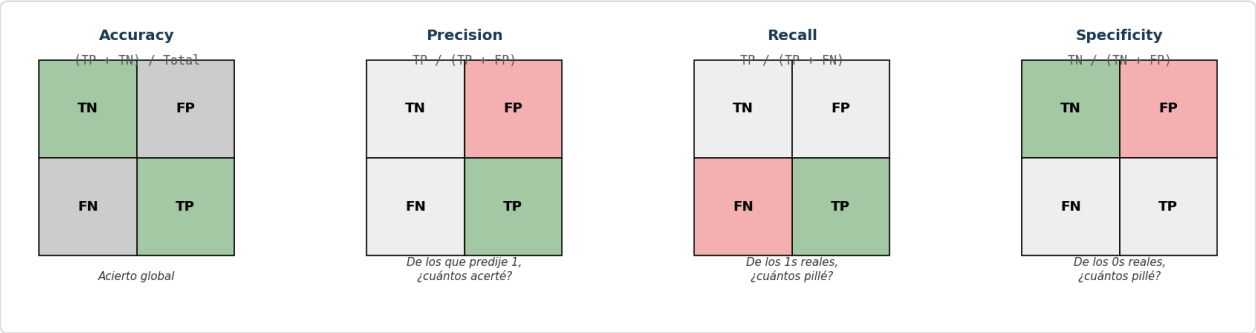
```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, predicciones)
print(cm)
# [[TN, FP],
#  [FN, TP]]
```

Para visualizarla bonita con seaborn:

```
import seaborn as sns
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
```

35. Precision, Recall, Specificity, Accuracy — las métricas derivadas

Todas las métricas que siguen son **combinaciones aritméticas** de las cuatro casillas de la matriz de confusión. Entendida la matriz, no hay que memorizar nada: se deducen.



Las cuatro métricas principales. En cada una, las celdas verdes son las que cuentan para numerador (lo que quieres maximizar) y las rojas las que restan (lo que quieres minimizar).

35.1 Accuracy (exactitud)

$$\text{Accuracy} = (\text{TP} + \text{TN}) \ / \ \text{Total}$$

Lo que ya vimos. Porcentaje total de aciertos.

35.2 Precision (precisión)

$$\text{Precision} = \text{TP} \ / \ (\text{TP} + \text{FP})$$

> **LA PREGUNTA QUE RESPONDE**
>
> De todos los casos que el modelo predijo como 1, ¿cuántos eran realmente 1?

Una precision de 0.90 significa que, cuando el modelo predice 1, el 90% de las veces acierta. Precision baja cuando hay muchos falsos positivos (el modelo grita "¡1!" demasiado a menudo).

Usa precision cuando los falsos positivos sean costosos.

35.3 Recall (sensibilidad)

$$\text{Recall} = \text{TP} \ / \ (\text{TP} + \text{FN})$$

> **LA PREGUNTA QUE RESPONDE**
>
> De todos los 1s reales que había, ¿cuántos fue capaz de pillar el modelo?

Un recall de 0.90 significa que el modelo detecta el 90% de los casos positivos reales. Recall bajo cuando hay muchos falsos negativos (se le escapan positivos).

Usa recall cuando los falsos negativos sean costosos.

35.4 Specificity (especificidad)

$$\text{Specificity} = \text{TN} \ / \ (\text{TN} + \text{FP})$$

El equivalente al recall pero para la clase negativa: de todos los 0s reales, ¿cuántos detectó correctamente? Útil cuando el foco está en clasificar bien los negativos.

36. Cuándo priorizar precision, recall o specificity

Esta es la parte práctica que tu profesor explicó con dos ejemplos muy claros. Hay que escoger la métrica adecuada según lo que te interese del problema real.

Clasificador de videos aptos para niños

Prioridad: PRECISION

- No puede colarse NINGÚN video inadecuado como apto.
- Un falso positivo (FP) = un video malo marcado como apto = ¡peligro para niños!
- Si perdemos algún video bueno clasificado como malo, no pasa nada grave.

→ MINIMIZAR FP → alta precision

Clasificador de ladrones en tienda

Prioridad: RECALL

- No puede escaparse NINGÚN ladrón (1) clasificado como inocente (0).
- Un falso negativo (FN) = un ladrón no detectado = grave.
- Si confundimos a algún cliente con ladrón, lo revisamos y a otra cosa.

→ MINIMIZAR FN → alto recall

Dos casos clásicos: clasificador de contenido para niños (prioriza precision porque no puede colarse nada malo) vs clasificador de ladrones (prioriza recall porque no se puede escapar ninguno).

36.1 Resumen de cuándo usar qué

Métrica	Minimizar	Cuándo usarla
Accuracy	Errores totales	Cuando las clases están balanceadas y todos los errores cuestan igual.
Precision	Falsos positivos (FP)	Cuando una falsa alarma es cara. Ej: filtro antispam (no quiero borrar emails importantes), video infantil (no quiero contenido inadecuado), mensajes publicitarios agresivos a clientes.
Recall	Falsos negativos (FN)	Cuando dejar escapar un positivo es peor. Ej: cáncer (no puedo fallar en detectarlo), ladrones, fraude bancario, fugas de seguridad.
Specificity	Falsos positivos (mirando desde la clase negativa)	Cuando el foco son los negativos correctamente clasificados.

36.2 El dilema precision vs recall

SON DOS FUERZAS OPUESTAS

Si bajas el umbral para detectar más 1s → recall sube, pero te cuelas más FP → precision baja.
Si subes el umbral para ser más selectivo → precision sube, pero se te escapan más 1s → recall baja.

Rara vez puedes maximizar ambas a la vez. Tienes que elegir cuál prima según el problema.

37. F1-Score: el compromiso entre precision y recall

A veces no hay una razón clara para priorizar precision sobre recall (o al revés) y quieres una métrica única que los combine. Para eso existe el **F1-Score**: la media armónica de precision y recall.

$$\mathrm{F1} = 2 \cdot (\mathrm{precision} \cdot \mathrm{recall}) \;/\; (\mathrm{precision} + \mathrm{recall})$$

POR QUÉ MEDIA ARMÓNICA (Y NO MEDIA ARITMÉTICA)

La media armónica castiga los valores desequilibrados. Si precision=0.9 y recall=0.1, la media aritmética sería 0.5 (parece decente); pero la media armónica (F1) sale 0.18 (refleja que el recall pésimo arruina el modelo). F1 solo es alto cuando precision Y recall son ambas altas.

F1 varía entre 0 (malo) y 1 (perfecto). Es la métrica preferida cuando las clases están desbalanceadas y te importan ambas por igual.

38. Predicción probabilística y el umbral

Como ya mencionamos en la Parte VI, los modelos de clasificación **no devuelven directamente una clase**: devuelven una probabilidad, y somos nosotros los que aplicamos un umbral para decidir.

El modelo devuelve probabilidades, no clases directas.
Aplicamos un umbral (por defecto 0.5) para decidir.

Persona	Edad	Fumador	Prob. clase 1	Prob. clase 0	Umbral 0.5 → Predicción
A	57	Sí	0.70	0.30	✓ 1 (enfermo)
B	32	No	0.10	0.90	✗ 0 (sano)
C	39	Sí	0.40	0.60	✗ 0 (sano)
D	60	Sí	0.85	0.15	✓ 1 (enfermo)

El modelo te da una probabilidad para cada clase. Con el umbral habitual de 0.5, la persona A (prob 0.70) se clasifica como 1, pero la C (prob 0.40) como 0 aunque esté muy cerca del límite.

38.1 Cómo mover el umbral

Mover el umbral cambia *drásticamente* la matriz de confusión y las métricas, **sin reentrenar el modelo**. Es una palanca que tienes gratis.

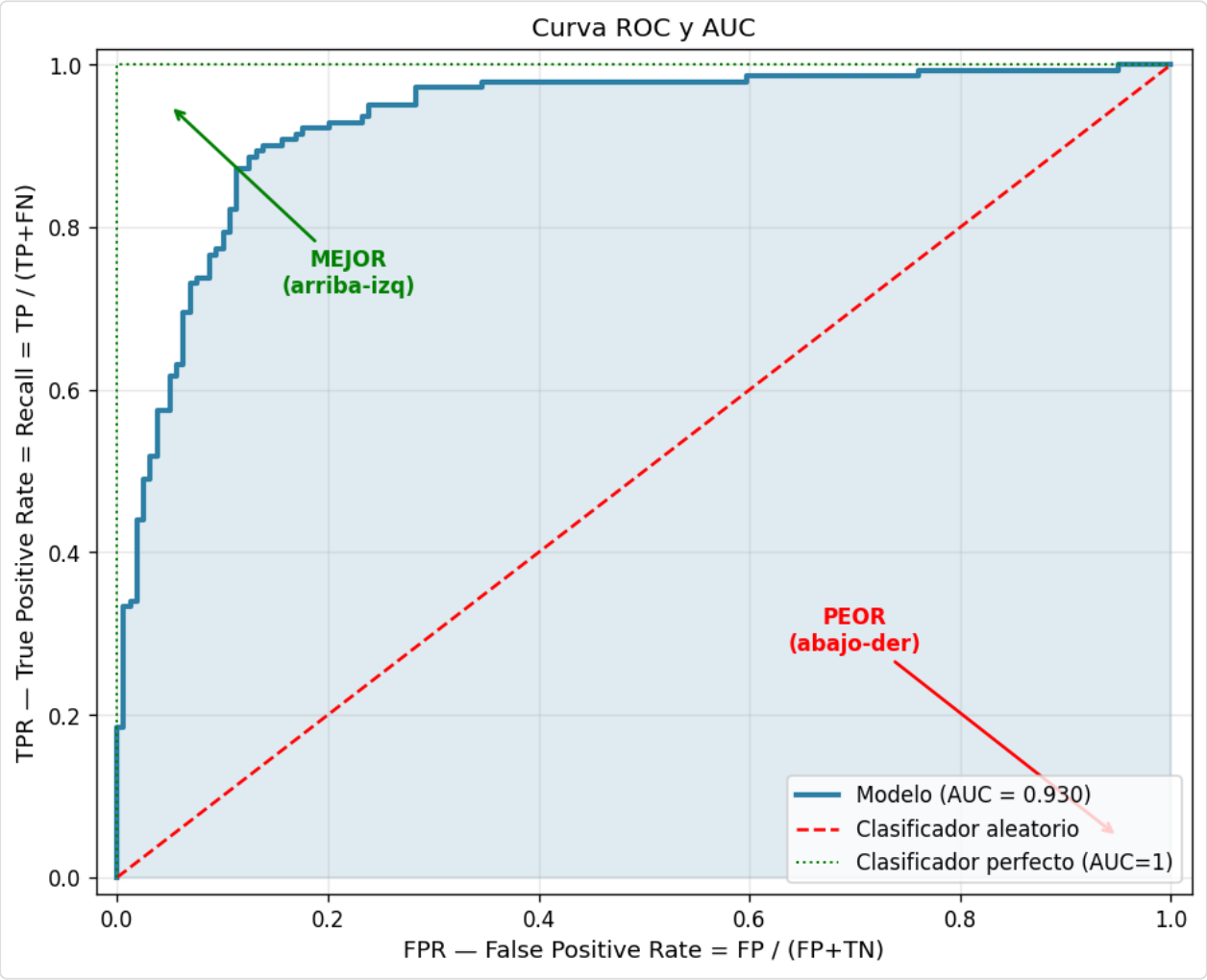
Umbral	Efecto	Quién sube / quién baja
< 0.5 (flexible con 1s)	Más FP, menos FN	↑ Recall, ↓ Precision
= 0.5 (por defecto)	Equilibrio	Comportamiento estándar
> 0.5 (restrictivo con 1s)	Menos FP, más FN	↓ Recall, ↑ Precision

39. Curva ROC y AUC

Si el umbral cambia las métricas y se puede poner en cualquier valor entre 0 y 1, ¿por qué no evaluar el modelo en todos los umbrales a la vez? Eso es justo lo que hace la curva ROC.

39.1 Qué es la curva ROC

La **curva ROC** (Receiver Operating Characteristic) es un gráfico que muestra el rendimiento del modelo para todos los umbrales posibles. Cada punto de la curva representa un umbral distinto.



Curva ROC del modelo entrenado en datos sintéticos. En el eje X, la tasa de falsos positivos (FPR): lo que queremos bajo. En el eje Y, la tasa de verdaderos positivos (TPR = recall): lo que queremos alto. La curva ideal sería la que sube vertical y luego va horizontal por arriba (AUC=1). La línea roja diagonal es un clasificador aleatorio (AUC=0.5). Nuestro modelo tiene AUC=0.93, muy bueno.

39.2 Cómo se construye la curva

1. Tomas todas las probabilidades que el modelo predijo para tus datos de test.
2. Pruebas muchos umbrales distintos (de 0 a 1).
3. Para cada umbral, calculas TPR (recall) y FPR (FP / (FP+TN)).
4. Cada par (FPR, TPR) es un punto en la curva.
5. Unir todos los puntos te da la curva ROC.

39.3 AUC: el área bajo la curva

AUC (Area Under the Curve) es el área debajo de la curva ROC. Es un número entre 0 y 1:

AUC	Interpretación
1.0	Clasificador perfecto (discrimina sin errores).
0.9 - 1.0	Excelente.
0.8 - 0.9	Bueno.
0.7 - 0.8	Aceptable.
0.5 - 0.7	Pobre.
= 0.5	Aleatorio. El modelo no sirve.
< 0.5	Peor que aleatorio. Algo ha fallado en el entrenamiento (o podrías invertir la predicción y tendrías AUC > 0.5).

POR QUÉ AUC ES TAN QUERIDO

AUC tiene dos ventajas grandes sobre las métricas que dependen de un umbral concreto:

1. **No depende del umbral**: evalúa al modelo en todos los umbrales a la vez. Si tu modelo tiene buen AUC, es robusto al cambio de umbral.
2. **Es robusto al desbalanceo de clases**: mucho menos afectado que accuracy.

39.4 Cómo interpretar zonas de la curva

- **Esquina superior izquierda (TPR=1, FPR=0):** el sueño. Detectas todos los 1s sin ningún FP. Cuanto más se acerque tu curva a esta esquina, mejor.
- **Esquina inferior izquierda (TPR=0, FPR=0):** umbral muy alto. El modelo casi nunca dice "1", así que no tiene FP, pero se deja todos los positivos (TPR=0).
- **Esquina superior derecha (TPR=1, FPR=1):** umbral muy bajo. El modelo dice "1" a todo. Pilla todos los positivos (TPR=1) pero también marca como positivo a todos los negativos (FPR=1).
- **Línea diagonal:** clasificador aleatorio. Si tu curva está sobre la diagonal, tu modelo es basura.

39.5 En sklearn

```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Probabilidades de la clase 1
y_scores = modelo.predict_proba(X_test)[:, 1]

# Calcular puntos de la curva ROC
fpr, tpr, thresholds = roc_curve(y_test, y_scores)
auc_valor = auc(fpr, tpr)

plt.plot(fpr, tpr, label=f"AUC = {auc_valor:.3f}")
plt.plot([0, 1], [0, 1], linestyle='--')  # diagonal
plt.xlabel("FPR"); plt.ylabel("TPR (Recall)")
plt.legend()
```

40. classification_report: el resumen que te da todo

En sklearn hay una función muy cómoda que te imprime un informe completo:

```
from sklearn.metrics import classification_report
print(classification_report(y_test, predicciones))
```

Te devuelve algo tipo:

```
              precision    recall  f1-score   support

           0       0.94      0.96      0.95       120
           1       0.90      0.85      0.87        80

    accuracy                           0.92       200
   macro avg       0.92      0.91      0.91       200
weighted avg       0.92      0.92      0.92       200
```

Te da precision, recall y F1 **para cada clase por separado**, más las medias. Es lo que se usa típicamente para reportar un modelo de clasificación.

Parte VIII
Feature Engineering para clasificación

En el notebook de Ad Click de tu clase (el de predecir si un usuario hace clic en un anuncio) viste técnicas que convierten datos crudos en features utilizables por el modelo. Estas técnicas no son específicas de clasificación — se usan igual en regresión — pero las veremos aquí porque es donde aparecieron. Son el 80% del trabajo real de un data analyst.

41. Feature Engineering: qué es y por qué importa

> **LA REGLA NO ESCRITA**
>
> "Un buen modelo con malas features nunca supera a un modelo sencillo con buenas features." El feature engineering es transformar los datos crudos en columnas que el modelo pueda aprovechar. Es creatividad, conocimiento del dominio y algo de código pandas.

Los problemas típicos con los datos crudos:

- Hay columnas de tipo texto libre ("Ad Topic Line") que un modelo no puede usar.
- Hay columnas categóricas ("Country", "City") con demasiados valores únicos.
- Hay fechas ("Timestamp") que contienen información rica (mes, día, hora) que no se extrae automáticamente.
- Hay columnas numéricas en escalas muy distintas que requieren escalado.

42. Codificación de variables categóricas

Los modelos de Machine Learning solo entienden números. Si una columna tiene categorías (Rojo, Azul, Verde), hay que convertirla. Dos técnicas principales:

LabelEncoder
Asigna un número entero a cada categoría
⚠ Introduce orden falso: 2 > 1 > 0

Original	LabelEncoder
Rojo	2
Azul	0
Verde	1
Rojo	2
Verde	1

OneHotEncoding (pd.get_dummies)
Una columna 0/1 por cada categoría
✓ Sin orden implícito, mejor para regresión

Original	Azul	Rojo	Verde
Rojo	0	1	0
Azul	1	0	0
Verde	0	0	1
Rojo	0	1	0
Verde	0	0	1

Dos formas de convertir categorías a números. LabelEncoder asigna un entero arbitrario a cada categoría. OneHotEncoding crea una columna 0/1 por cada categoría. Cada una tiene su momento adecuado.

42.1 LabelEncoder

Asigna un entero a cada categoría: {Rojo: 0, Azul: 1, Verde: 2}. Sencillo pero **peligroso**: introduce un orden artificial entre categorías. El modelo puede pensar que "Verde (2) > Azul (1) > Rojo (0)", cuando no existe tal orden.

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
df['Country_encoded'] = encoder.fit_transform(df['Country'])
```

CUÁNDO NO USAR LABELENCODER

No lo uses en variables categóricas sin orden natural (colores, países, ciudades). El modelo interpretará los números como orden y tomará decisiones basadas en eso.

Sí puedes usarlo para variables *ordinales* (tienen orden natural): bajo/medio/alto, malo/regular/bueno/excelente.

42.2 OneHotEncoding

Crea una columna binaria (0 o 1) por cada categoría. Una fila solo tiene un 1 en la columna que corresponde a su categoría, el resto son 0. Sin orden implícito → el modelo trata todas las categorías por igual.

```
# Opción 1: pandas
df_encoded = pd.get_dummies(df['Country'])

# Opción 2: sklearn
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False)
country_encoded = ohe.fit_transform(df[['Country']])
```

CUÁNDO USAR QUÉ

Situación	Técnica
Categoría con orden natural (bajo/medio/alto)	LabelEncoder
Categoría sin orden, pocas clases (< 15)	OneHotEncoding
Categoría sin orden, muchas clases (países, ciudades)	Cuidado: OneHot puede crear cientos de columnas. Alternativas: target encoding, frequency encoding, agrupar categorías raras.

42.3 El problema de la alta cardinalidad

En el notebook de Ad Click, la columna `City` tenía casi tantos valores únicos como filas (cardinalidad muy alta). Aplicar OneHotEncoding habría creado cientos de columnas casi vacías, inflando el dataset sin aportar señal. En esos casos conviene:

- **Agrupar categorías raras** en una única categoría "Otros".
- **Target encoding**: reemplazar cada ciudad por la probabilidad media de clic en esa ciudad.
- **Frequency encoding**: reemplazar cada ciudad por el número de veces que aparece.
- Simplemente **descartar la columna** si no aporta.

43. Feature engineering de fechas

Una columna Timestamp contiene muchísima información potencial que el modelo no extrae por defecto. Hay que desglosarla manualmente.

Feature engineering de fechas
De un único Timestamp, extraemos 4 features nuevas usando .dt

Timestamp original	month	day	weekday	hour
2024-03-15 14:23:00	3	15	4 (vie)	14
2024-07-20 09:45:00	7	20	5 (sáb)	9
2024-11-05 21:10:00	11	5	1 (mar)	21

De un Timestamp sacamos 4 features nuevas: mes, día, día de la semana y hora. Cada una puede correlacionar con el target (ej. en Ad Click los clics varían por hora del día).

El código pandas típico:

```
# Primero aseguramos que la columna es tipo datetime
df['Timestamp'] = pd.to_datetime(df['Timestamp'])

# Extraemos componentes usando el accesor .dt
df['month']   = df['Timestamp'].dt.month       # 1 a 12
df['day']     = df['Timestamp'].dt.day         # 1 a 31
df['weekday'] = df['Timestamp'].dt.weekday     # 0=lun, 6=dom
df['hour']    = df['Timestamp'].dt.hour        # 0 a 23
df['year']    = df['Timestamp'].dt.year        # año
```

43.1 Por qué funcionan las features de fecha

Muchos fenómenos tienen estacionalidad: compras en diciembre, tráfico web en horas laborales, fines de semana con patrones distintos. Si no desglosas el timestamp, el modelo no puede ver esos patrones.

44. Feature engineering de texto

Cuando tienes una columna de texto libre (como "Ad Topic Line"), no puedes pasarla al modelo directamente. Hay que crear features binarias o numéricas a partir de ella.

44.1 Búsqueda de palabras clave

Si sospechas que ciertas palabras están asociadas al target, creas una feature binaria:

```
import numpy as np

# ¿Contiene la palabra "solution"?
df['Topic_solution'] = np.where(
    df['Ad Topic Line'].str.contains('solution', case=False),
    1, 0
)
```

44.2 Frecuencia de palabras

Para identificar palabras candidatas, vemos las más frecuentes:

```
lista_palabras = []
for topic in df['Ad Topic Line']:
    for palabra in topic.split(" "):
        lista_palabras.append(palabra.lower())

pd.Series(lista_palabras).value_counts().head(20)
```

44.3 Técnicas más avanzadas

Para casos con mucho texto (reviews, descripciones largas), hay técnicas más sofisticadas que no vimos en clase pero conviene conocer por nombre:

- **Bag of Words (BoW)**: crea una columna por cada palabra del vocabulario.
- **TF-IDF**: versión ponderada que castiga palabras demasiado comunes.
- **Word embeddings**: representaciones vectoriales de palabras que capturan semántica (más avanzado).

45. Pipeline completo de feature engineering

Uniendo todo, el flujo típico que viste en el notebook Ad Click sería:

1. **Análisis exploratorio** (EDA): ver qué columnas tienes, qué tipos son, nulos, distribuciones.
2. **Limpieza**: corregir errores tipográficos ("newpaper" → "newspaper"), convertir tipos de dato, tratar nulos.
3. **Feature engineering de texto**: extraer features binarias de columnas de texto libre.
4. **Feature engineering de fechas**: desglosar timestamps en mes, día, hora, etc.
5. **Codificación de categóricas**: LabelEncoder u OneHotEncoding según el caso.
6. **Train/test split**: ANTES de escalar, para evitar contaminación.
7. **Escalado**: StandardScaler o MinMaxScaler ajustado SOLO con train.
8. **Entrenar el modelo**: logística, regresión, o lo que corresponda.
9. **Evaluar**: con las métricas apropiadas al problema.

> **REGLA DE ORO DEL FEATURE ENGINEERING**
>
> Piensa siempre **como analista de dominio, no como programador**. ¿Qué variables afectarían de verdad al comportamiento que intentas predecir? Esa intuición es la que te dice qué features inventar. El código es lo fácil; saber qué extraer es lo valioso.

Parte IX
Cheat sheet y glosario final

Material de consulta rápida. Úsalo cuando te falten referencias durante ejercicios, entrevistas o estudio. No sustituye a las partes anteriores pero condensa lo más práctico.

46. Pipeline típico de un problema de regresión

1. **EDA:** `df.head()`, `df.info()`, `df.describe()`, `df.isnull().sum()`.
2. **Visualización:** histogramas del target, heatmap de correlaciones, scatter plots.
3. **Feature engineering:** limpiar nulos, codificar categóricas, desglosar fechas.
4. **Selección de features:** eliminar las que correlacionen mucho entre sí, quedarse con las más correlacionadas con el target.
5. **Train/test split:** `train_test_split(X, y, test_size=0.2, random_state=42)`.
6. **Escalado:** `StandardScaler` ajustado con fit en train, transform en train y test.
7. **Baseline:** entrenar `LinearRegression` sin más como punto de partida.
8. **Iteración:** probar `Ridge` / `Lasso` / `ElasticNet`, buscar alpha con CV, probar polinómica de grado 2 o 3.
9. **Evaluación:** MAE, RMSE, R^2 en test. Histograma de residuos para ver si son normales.
10. **Validación cruzada:** `cross_val_score` para confirmar que el modelo es estable.
11. **Reporte:** métricas finales, gráfico y vs $\hat{y}$, interpretación de coeficientes.

47. Pipeline típico de un problema de clasificación

1. **EDA:** ver balanceo de clases con `df['target'].value_counts()`.
2. **Feature engineering:** igual que en regresión (categóricas, fechas, texto).
3. **Train/test split** con `stratify=y` para mantener el balance de clases.
4. **Escalado:** obligatorio si usas logística con regularización.
5. **Baseline:** `LogisticRegression(max_iter=1000)`.
6. **Evaluación:** matriz de confusión, precision, recall, F1, curva ROC, AUC.
7. **Ajustar umbral** según lo que priorices (precision vs recall).
8. **Iteración**: probar otros modelos (árboles, RF), tunear hiperparámetros, resampling si hay desbalance.
9. **Reporte:** `classification_report`, curva ROC, interpretación del negocio.

48. Cheat sheet de métricas

48.1 Métricas de regresión

Métrica	Fórmula	Unidades	Rango	Mejor cuando
MAE	media(|y−ŷ|)	las del target	≥ 0	Interpretación simple
MSE	media((y−ŷ)²)	al cuadrado	≥ 0	Quieres castigar errores grandes
RMSE	√MSE	las del target	≥ 0	Reporte estándar, castiga outliers
MAPE	media(|y−ŷ|/|y|)·100	%	≥ 0	Error relativo, independiente de escala
R^2	$1 - SS_{res}/SS_{tot}$	adimensional	(−∞, 1]	¿El modelo bate a la media?

48.2 Métricas de clasificación binaria

Métrica	Fórmula	Minimiza	Usar cuando
Accuracy	(TP+TN)/Total	Errores globales	Clases balanceadas
Precision	TP/(TP+FP)	FP (falsas alarmas)	FP caros (spam, contenido infantil)
Recall (TPR)	TP/(TP+FN)	FN (se escapan 1s)	FN caros (cáncer, ladrones)
Specificity	TN/(TN+FP)	FP desde los 0s	Foco en negativos correctos
F1	2·P·R/(P+R)	Desequilibrio P-R	Balance P y R, clases desbalanceadas
AUC-ROC	Área bajo curva ROC	N/A (global)	Comparar modelos sin elegir umbral

49. Cheat sheet de modelos sklearn

```
# Regresión
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
LinearRegression()                          # sin regularización
Ridge(alpha=1.0)                           # L2
Lasso(alpha=0.1, max_iter=10000)           # L1 (feature selection)
ElasticNet(alpha=0.1, l1_ratio=0.5)         # mezcla L1+L2

# Polinómica (la haces con LinearRegression sobre features polinómicas)
from sklearn.preprocessing import PolynomialFeatures
pf = PolynomialFeatures(degree=2)

# Clasificación
from sklearn.linear_model import LogisticRegression
LogisticRegression(max_iter=1000)

# Preprocesado
from sklearn.preprocessing
import StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder

# Split y validación
from sklearn.model_selection import train_test_split, cross_val_score

# Métricas
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,              # regre-
sión
    accuracy_score, precision_score, recall_score, f1_score,        # clasif.
    confusion_matrix, classification_report, roc_curve, roc_auc_score
)
```

50. Glosario alfabético

Accuracy

Porcentaje de predicciones correctas sobre el total. Engaña con clases desbalanceadas.

Alpha (α)

Hiperparámetro de regularización. Controla cuánto se penalizan los coeficientes grandes en Ridge, Lasso y Elastic Net.

AUC (Area Under the Curve)

Área bajo la curva ROC. Entre 0 y 1. Mide la capacidad del modelo de separar clases, independientemente del umbral.

Bias (sesgo)

Error por modelo demasiado simple. Causa underfitting.

Classification

Predecir una categoría (binaria o multiclase).

Coeficiente (β, weight, w)

Cada uno de los parámetros que multiplican las features en la ecuación del modelo. Se aprenden durante el entrenamiento.

Convergencia

Estado en el que Gradient Descent deja de mejorar: el modelo ha encontrado el mínimo (o se ha estancado).

Correlación de Pearson (r)

Medida lineal entre dos variables. Rango [−1, 1]. Solo detecta relaciones lineales.

Cross-validation (validación cruzada)

Técnica que divide los datos en K pliegues y entrena/evalúa K veces rotando qué pliegue va a test. Da una estimación más robusta del rendimiento.

Cross-entropy / log-loss

Función de costes usada en regresión logística para medir qué tan lejos están las probabilidades predichas de la clase real.

Elastic Net

Regularización que combina L1 y L2. Controlada por dos hiperparámetros: alpha y l1_ratio.

Feature

Cada variable predictora. También llamada *predictor*, *variable independiente*, *X*.

Feature engineering

Proceso de transformar datos crudos en features aprovechables por el modelo.

Feature importance

Ranking de las features por su contribución al modelo. Se calcula estandarizando las features y mirando el valor absoluto de los coeficientes.

F1-score

Media armónica de precision y recall. Rango [0, 1]. Balancea ambas métricas.

FPR (False Positive Rate)

Tasa de falsos positivos. FP / (FP + TN). Es el eje X de la curva ROC.

Gradient Descent

Algoritmo iterativo que busca los parámetros que minimizan la función de costes siguiendo la pendiente descendente.

Hiperparámetro

Configuración del modelo que decides tú antes de entrenar (learning rate, alpha, grado del polinomio). No lo aprende el modelo.

Intercepto (β_0)

Término independiente de la ecuación. Valor predicho cuando todas las features valen 0.

L1 (norma L1)

Suma de valores absolutos de los coeficientes. Penalización usada por Lasso.

L2 (norma L2)

Suma de cuadrados de los coeficientes. Penalización usada por Ridge.

LabelEncoder

Convierte categorías en enteros arbitrarios. Introduce orden artificial.

Lasso

Regresión con regularización L1. Capaz de poner coeficientes a 0 → hace feature selection.

Learning rate (α en gradient descent)

Tamaño del paso en cada iteración. Demasiado grande: oscila. Demasiado pequeño: tarda.

Log-odds (logit)

Logaritmo del cociente P(Y=1)/P(Y=0). Es la cantidad que la regresión logística modela como combinación lineal de features.

Logistic Regression

A pesar del nombre, es un modelo de CLASIFICACIÓN. Aplica la sigmoide a una combinación lineal de features para obtener probabilidades.

MAE (Mean Absolute Error)

Media del valor absoluto de los errores. En las unidades del target.

MAPE (Mean Absolute Percentage Error)

Error medio como porcentaje del valor real. Útil para comparar escalas distintas.

Matriz de confusión

Tabla 2x2 (en binaria) que cuenta TP, TN, FP, FN. Base de todas las métricas de clasificación.

MinMaxScaler

Escalador que lleva los valores al rango [0, 1].

MSE (Mean Squared Error)

Media de los errores al cuadrado. Castiga más los errores grandes.

Multicolinealidad

Cuando dos features están muy correlacionadas. Vuelve inestables los coeficientes del modelo.

OneHotEncoding

Codificación que crea una columna binaria por cada categoría. Sin orden implícito.

Overfitting

Modelo demasiado complejo que memoriza el train en lugar de generalizar. Buen rendimiento en train, malo en test.

One-vs-Rest (OvR, OvA)

Estrategia multiclase que entrena N modelos binarios, uno por clase. La clase ganadora es la de mayor probabilidad.

p-valor

Probabilidad de observar un resultado como el obtenido si en la realidad no hubiera efecto. < 0.05 → estadísticamente significativo.

PolynomialFeatures

Transformador de sklearn que genera potencias y productos cruzados de las features originales para la regresión polinómica.

Precision

De los predichos como 1, qué fracción era realmente 1. TP / (TP + FP). Mejora bajando FP.

R^2 (coeficiente de determinación)

Fracción de varianza del target que explica el modelo respecto a predecir siempre la media. Rango $(-\infty, 1]$. Negativo = peor que la media.

Recall (sensibilidad, TPR)

De los 1s reales, qué fracción detectó el modelo. TP / (TP + FN). Mejora bajando FN.

Regresión

Predecir un número continuo.

Regularización

Técnica que añade una penalización a la función de costes para evitar overfitting.

Residuo

Diferencia entre valor real y predicho para un ejemplo. También se llama error.

Ridge

Regresión con regularización L2. Reduce coeficientes pero no los pone a cero.

RMSE (Root Mean Squared Error)

Raíz del MSE. En las unidades del target, castiga errores grandes.

ROC (Receiver Operating Characteristic)

Curva que muestra TPR vs FPR para todos los umbrales. El área bajo la curva (AUC) mide el rendimiento global.

Sigmoide

Función $\sigma(z) = 1/(1+e^{-z})$ que transforma cualquier número en un valor [0, 1]. Corazón de la regresión logística.

Specificity

De los 0s reales, qué fracción detectó el modelo como 0. TN / (TN + FP).

StandardScaler

Escalador que transforma cada feature a media 0, desviación típica 1.

Target

Variable que queremos predecir. También llamada *variable dependiente*, *respuesta*, *Y*.

Threshold (umbral)

Probabilidad de corte que decide si una predicción se asigna a la clase 1 o 0. Por defecto 0.5.

TPR (True Positive Rate)

Lo mismo que recall. Eje Y de la curva ROC.

Train / test split

División de los datos en conjunto de entrenamiento (para aprender) y conjunto de prueba (para evaluar honestamente).

Underfitting

Modelo demasiado simple, que no captura los patrones. Mal rendimiento en train y test.

Variance (varianza)

Error por modelo demasiado sensible a los datos de entrenamiento. Causa overfitting.

51. Palabras finales

Has llegado al final. Esta guía cubre todo el bloque de Machine Learning que necesitabas ponerte al día. Si la has leído entera, tienes una base sólida. Si la usas de referencia mientras resuelves ejercicios, acabarás interiorizándola. Ambos caminos son válidos.

El objetivo no era memorizar fórmulas, sino *entender* qué hace cada cosa y por qué. Con eso, puedes defender cualquier resultado que obtengas y, sobre todo, diagnosticar por qué un modelo no funciona cuando se tuerce. Esa es la habilidad real de un data analyst.

Sigue practicando los notebooks. Cada uno es un ejercicio completo de este pipeline. Cuando veas una técnica nueva en clase, vuelve a esta guía, busca el concepto base y conecta lo nuevo con lo que ya sabes. Así se construye expertise de verdad.